TFG: IA Explicable sobre datos transcriptómicos de cáncer basados en oncogenes

Alberto

2026-06-17

Contents

Introducción

Este documento recoge los resultados del análisis de datos bulk RNA-seq de cáncer mediante técnicas de Machine Learning. Partimos del tutorial "Artificial intelligence in bulk and single-cell RNA-sequencing data to foster precision oncology" (International Journal of Molecular Sciences, 2021).

El objetivo es clasificar muestras de tejido tumoral y sano usando Random Forest y SVM, extraer los genes más importantes, y aplicar técnicas de IA explicable (SHAP, LIME, PDP).

Referencias:

- Tutorial: https://www.ceredalab.com/AI/
- Datos RNA-seq: Wang et al. (2018), Scientific Data. DOI: 10.1038/sdata.2018.61
- Repositorio datos: https://github.com/mskcc/RNAseqDB
- Genes de cáncer: NCG 7.1. Repana et al. (2019), Genome Biology. DOI: 10.1186/s13059-018-1612-0

Carga de librerías y datos

```
library(caret)
library(randomForest)
library(gridExtra)
library(e1071)
library(kernlab)
library(Rtsne)
library(ggplot2)
```

Datos de expresión génica

Los datos provienen de dos proyectos públicos: TCGA (muestras tumorales) y GTEx (tejido sano). Se trabaja con cáncer de próstata (PRAD) y pulmón (LUAD). Se parte de cuatro ficheros de expresión (uno por cada combinación de tejido y condición), que en conjunto suman 1348 muestras descritas por unos 19 000 genes.

```
normal_A = read.table('sources/prostate-rsem-fpkm-gtex.txt.gz', header = T)
normal_B = read.table('sources/lung-rsem-fpkm-gtex.txt.gz', header = T)
tumor_A = read.table('sources/prad-rsem-fpkm-tcga-t.txt.gz', header = T)
tumor_B = read.table('sources/luad-rsem-fpkm-tcga-t.txt.gz', header = T)

cat("Tumor próstata (PRAD):", nrow(tumor_A), "genes x", ncol(tumor_A)-2, "muestras\n")
```

```
## Tumor próstata (PRAD): 19046 genes x 426 muestras
```

```
cat("Tumor pulmón (LUAD):", nrow(tumor_B), "genes x", ncol(tumor_B)-2, "muestras\n")
```

```
## Tumor pulmón (LUAD): 19648 genes x 503 muestras
```

```
cat("Normal próstata:", nrow(normal_A), "genes x", ncol(normal_A)-2, "muestras\n")
```

```
## Normal próstata: 19046 genes x 106 muestras
```

```r
cat("Normal pulmón:", nrow(normal_B), "genes x", ncol(normal_B)-2, "muestras\n")
```

```
## Normal pulmón: 19648 genes x 313 muestras
```

Limpieza del dataset de genes de cáncer

El fichero original descargado de NCG 7.1 contiene 8886 filas (genes repetidos en varios estudios) y 14 columnas. Lo limpiamos para quedarnos solo con oncogenes y TSGs confirmados.

```r
ncg7 = read.delim('sources/NCG_cancerdrivers_annotation_supporting_evidence.tsv', header = T, sep = '\t
cat("Fichero original:", nrow(ncg7), "filas,", ncol(ncg7), "columnas\n")
```

```
## Fichero original: 8886 filas, 14 columnas
```

```r
cat("Genes únicos:", length(unique(ncg7$entrez)), "\n")
```

```
## Genes únicos: 3347
```

```r
# Paso 1: Solo oncogenes y TSGs confirmados
ncg7_filtrado = subset(ncg7, NCG_oncogene == 1 | NCG_tsg == 1)
cat("Filas con oncogene o TSG confirmado:", nrow(ncg7_filtrado), "\n")
```

```
## Filas con oncogene o TSG confirmado: 4582
```

```r
# Paso 2: Eliminar duplicados
ncg7_unicos = ncg7_filtrado[!duplicated(ncg7_filtrado$entrez),]
cat("Genes únicos tras eliminar duplicados:", nrow(ncg7_unicos), "\n")
```

```
## Genes únicos tras eliminar duplicados: 510
```

```r
# Paso 3: Crear columna type
ncg7_unicos$type = ifelse(ncg7_unicos$NCG_oncogene == 1 & ncg7_unicos$NCG_tsg == 1, "oncogene,tsg",
                   ifelse(ncg7_unicos$NCG_oncogene == 1, "oncogene", "tsg"))
cat("Oncogenes:", sum(ncg7_unicos$type == "oncogene"), "\n")
```

```
## Oncogenes: 256
```

```r
cat("TSGs:", sum(ncg7_unicos$type == "tsg"), "\n")
```

```
## TSGs: 254
```

```r
# Paso 4: Formato final
cancer_genes = data.frame(
  symbol = ncg7_unicos$symbol,
  entrez = ncg7_unicos$entrez,
  type = ncg7_unicos$type
)
cat("Total genes:", nrow(cancer_genes), "\n")
```

```
## Total genes: 510
```

Filtrado de datos por genes de cáncer

```
tumor_A = subset(tumor_A, Entrez_Gene_Id %in% cancer_genes$entrez)
rep = names(table(tumor_A$Entrez_Gene_Id)[which(table(tumor_A$Entrez_Gene_Id)>1)])
tumor_A = subset(tumor_A, !Entrez_Gene_Id %in% rep | Hugo_Symbol %in% subset(cancer_genes, entrez %in% 
rownames(tumor_A) = tumor_A$Entrez_Gene_Id

tumor_B = subset(tumor_B, Entrez_Gene_Id %in% cancer_genes$entrez)
rep = names(table(tumor_B$Entrez_Gene_Id)[which(table(tumor_B$Entrez_Gene_Id)>1)])
tumor_B = subset(tumor_B, !Entrez_Gene_Id %in% rep | Hugo_Symbol %in% subset(cancer_genes, entrez %in% 
rownames(tumor_B) = tumor_B$Entrez_Gene_Id

normal_A = subset(normal_A, Entrez_Gene_Id %in% cancer_genes$entrez)
rep = names(table(normal_A$Entrez_Gene_Id)[which(table(normal_A$Entrez_Gene_Id)>1)])
normal_A = subset(normal_A, !Entrez_Gene_Id %in% rep | Hugo_Symbol %in% subset(cancer_genes, entrez %in
rownames(normal_A) = normal_A$Entrez_Gene_Id

normal_B = subset(normal_B, Entrez_Gene_Id %in% cancer_genes$entrez)
rep = names(table(normal_B$Entrez_Gene_Id)[which(table(normal_B$Entrez_Gene_Id)>1)])
normal_B = subset(normal_B, !Entrez_Gene_Id %in% rep | Hugo_Symbol %in% subset(cancer_genes, entrez %in
rownames(normal_B) = normal_B$Entrez_Gene_Id

cat("Genes tras filtrado:", nrow(tumor_A), "\n")
```

```
## Genes tras filtrado: 506
```

Fase 1: Reproducción del tutorial

Separación train/test (75%/25%)

```
set.seed(900808)
tumor_A_train = tumor_A[,c("Hugo_Symbol","Entrez_Gene_Id", sample(x = colnames(tumor_A)[3:ncol(tumor_A)]
tumor_A_test = cbind(tumor_A[,1:2], tumor_A[,!colnames(tumor_A) %in% colnames(tumor_A_train)])
tumor_B_train = tumor_B[,c("Hugo_Symbol","Entrez_Gene_Id", sample(x = colnames(tumor_B)[3:ncol(tumor_B)]
tumor_B_test = cbind(tumor_B[,1:2], tumor_B[,!colnames(tumor_B) %in% colnames(tumor_B_train)])
normal_A_train = normal_A[,c("Hugo_Symbol","Entrez_Gene_Id", sample(x = colnames(normal_A)[3:ncol(normal
normal_A_test = cbind(normal_A[,1:2], normal_A[,!colnames(normal_A) %in% colnames(normal_A_train)])
normal_B_train = normal_B[,c("Hugo_Symbol","Entrez_Gene_Id", sample(x = colnames(normal_B)[3:ncol(normal
normal_B_test = cbind(normal_B[,1:2], normal_B[,!colnames(normal_B) %in% colnames(normal_B_train)])
```

Clasificación Tumor vs Normal

Construcción de matrices

```
tumor_A_train = subset(tumor_A_train, Entrez_Gene_Id %in% tumor_B_train$Entrez_Gene_Id)
tumor_B_train = subset(tumor_B_train, Entrez_Gene_Id %in% tumor_A_train$Entrez_Gene_Id)
tumor_train_set = as.data.frame(t(cbind(tumor_A_train[,3:ncol(tumor_A_train)], tumor_B_train[,3:ncol(tu
```

```r
normal_A_train = subset(normal_A_train, Entrez_Gene_Id %in% normal_B_train$Entrez_Gene_Id)
normal_B_train = subset(normal_B_train, Entrez_Gene_Id %in% normal_A_train$Entrez_Gene_Id)
normal_train_set = as.data.frame(t(cbind(normal_A_train[,3:ncol(normal_A_train)], normal_B_train[,3:nco

ncol(tumor_train_set) == ncol(normal_train_set)
```

```
## [1] TRUE
```

```r
sum(colnames(tumor_train_set)==colnames(normal_train_set))/ncol(normal_train_set)
```

```
## [1] 1
```

```r
train_set = rbind(tumor_train_set, normal_train_set)
colnames(train_set) = paste0('g_', colnames(train_set)[1:(ncol(train_set))])
train_set$type = factor(c(rep('Tumor', nrow(tumor_train_set)), rep('Normal', nrow(normal_train_set))), l

tumor_A_test = subset(tumor_A_test, Entrez_Gene_Id %in% tumor_B_test$Entrez_Gene_Id)
tumor_B_test = subset(tumor_B_test, Entrez_Gene_Id %in% tumor_A_test$Entrez_Gene_Id)
tumor_test_set = as.data.frame(t(cbind(tumor_A_test[,3:ncol(tumor_A_test)], tumor_B_test[,3:ncol(tumor_

normal_A_test = subset(normal_A_test, Entrez_Gene_Id %in% normal_B_test$Entrez_Gene_Id)
normal_B_test = subset(normal_B_test, Entrez_Gene_Id %in% normal_A_test$Entrez_Gene_Id)
normal_test_set = as.data.frame(t(cbind(normal_A_test[,3:ncol(normal_A_test)], normal_B_test[,3:ncol(no

ncol(tumor_test_set) == ncol(normal_test_set)
```

```
## [1] TRUE
```

```r
sum(colnames(tumor_test_set)==colnames(normal_test_set))/ncol(normal_test_set)
```

```
## [1] 1
```

```r
test_set = rbind(tumor_test_set, normal_test_set)
colnames(test_set) = paste0('g_', colnames(test_set)[1:(ncol(test_set))])
test_set$type = factor(c(rep('Tumor', nrow(tumor_test_set)), rep('Normal', nrow(normal_test_set))), leve

cat("Training set:", nrow(train_set), "muestras x", ncol(train_set)-1, "genes\n")
```

```
## Training set: 1012 muestras x 506 genes
```

```r
cat("Test set:", nrow(test_set), "muestras x", ncol(test_set)-1, "genes\n")
```

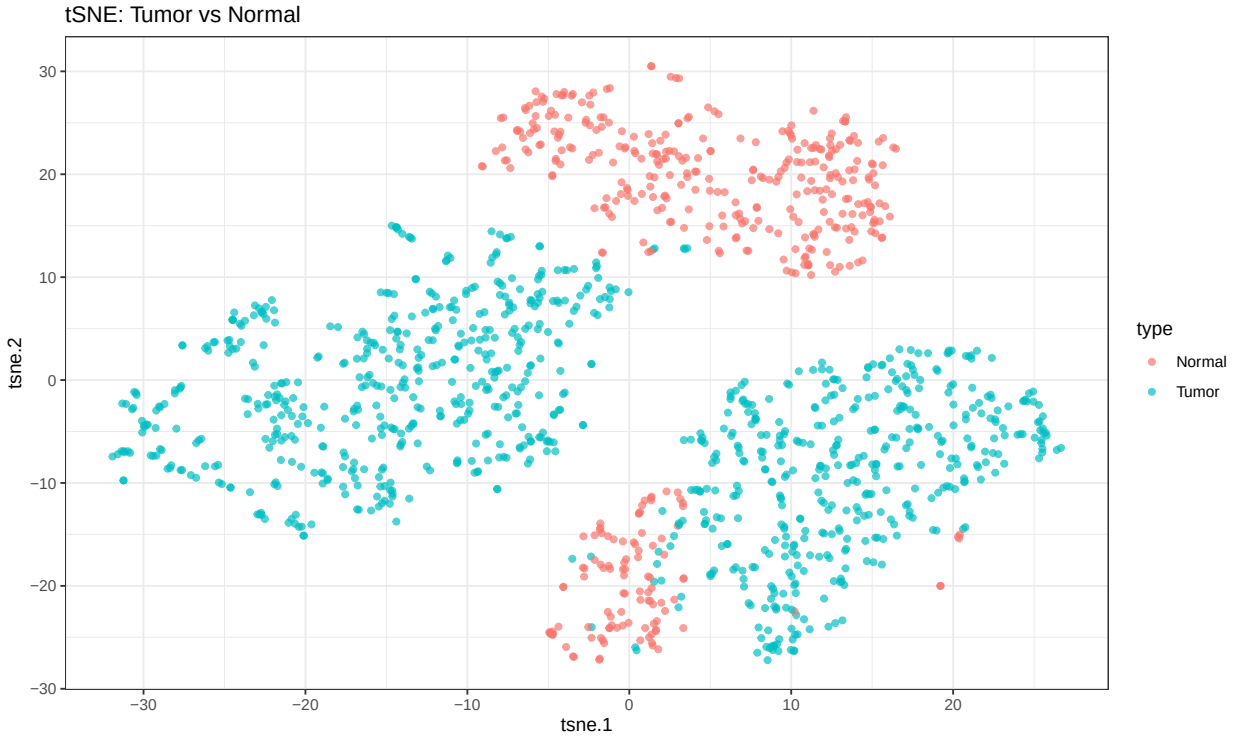
```
## Test set: 336 muestras x 506 genes
```

Preprocesamiento

```r
tmp = rbind(train_set, test_set)
all_data = tmp[, 1:(ncol(tmp)-1)]
all_data = predict(preProcess(all_data, method = c("center", "scale", "YeoJohnson", "nzv")), all_data)
all_data$type = tmp$type
train_set = all_data[1:nrow(train_set),]
test_set = all_data[(nrow(train_set)+1):(nrow(train_set)+nrow(test_set)),]
```

Visualización tSNE

```r
set.seed(94512)
tsne <- Rtsne(all_data[, colnames(all_data) != 'type'], dims = 2, perplexity = 15, verbose = FALSE, max_
tsne.df = as.data.frame(tsne$Y)
rownames(tsne.df) = rownames(all_data)
colnames(tsne.df) = c('tsne.1', 'tsne.2')
tsne.df$type = all_data$type

ggplot(tsne.df, aes(x = tsne.1, y = tsne.2, color = type)) +
  geom_point(alpha = 0.7) +
  ggtitle("tSNE: Tumor vs Normal") +
  theme_bw()
```



Se observa una separación clara entre muestras tumorales y normales.

Entrenamiento Random Forest

```
sel = colnames(train_set[1:(ncol(train_set)-1)])
formula <- as.formula(paste("type ~ ", paste(sel, collapse = "+")))
to_forest = train_set
train_control <- trainControl(method = "cv", number = 10, classProbs = TRUE, summaryFunction = twoClass

set.seed(900808)
rf2 <- train(formula, data = to_forest, method = "rf", tuneLength = 10, importance = T, trControl = tra
rf2
```

```
## Random Forest
##
## 1012 samples
##  506 predictor
##    2 classes: 'Normal', 'Tumor'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 911, 911, 911, 911, 910, 910, ...
## Resampling results across tuning parameters:
##
##   mtry  ROC        Sens       Spec
##     2   0.9984936  0.9140121  0.9957143
##     3   0.9986354  0.9267137  0.9928364
##     6   0.9985467  0.9489919  0.9913872
##    12   0.9985710  0.9553427  0.9899586
##    23   0.9985029  0.9584677  0.9871014
##    43   0.9984388  0.9586694  0.9885300
##    80   0.9982116  0.9555444  0.9899586
##   147   0.9973341  0.9587702  0.9885300
##   273   0.9968156  0.9526210  0.9856522
##   506   0.9939359  0.9204637  0.9813251
##
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 3.
```

```
postResample(predict(rf2, newdata = to_forest), to_forest$type)
```

```
## Accuracy    Kappa
##        1        1
```

Entrenamiento SVM

```
to_svm = train_set
set.seed(900808)
svm <- train(formula, data = to_svm, method = "svmRadial", tuneLength = 10, importance = T, trControl =
svm
```

```
## Support Vector Machines with Radial Basis Function Kernel
```

```
##
## 1012 samples
##  506 predictor
##    2 classes: 'Normal', 'Tumor'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 911, 911, 911, 911, 910, 910, ...
## Resampling results across tuning parameters:
##
##   C       ROC        Sens       Spec
##     0.25  0.9977258  0.9809476  0.9785300
##     0.50  0.9989138  0.9840726  0.9799586
##     1.00  0.9994118  0.9778226  0.9871222
##     2.00  0.9997307  0.9873992  0.9914079
##     4.00  0.9997307  0.9968750  0.9942857
##     8.00  0.9996400  0.9968750  0.9942857
##    16.00  0.9996400  0.9968750  0.9942857
##    32.00  0.9996400  0.9968750  0.9928571
##    64.00  0.9996400  0.9968750  0.9942857
##   128.00  0.9996400  0.9968750  0.9928571
##
## Tuning parameter 'sigma' was held constant at a value of 0.002209426
## ROC was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.002209426 and C = 2.
```

```
postResample(predict(svm, newdata = to_svm), to_svm$type)
```

```
## Accuracy    Kappa
##        1        1
```

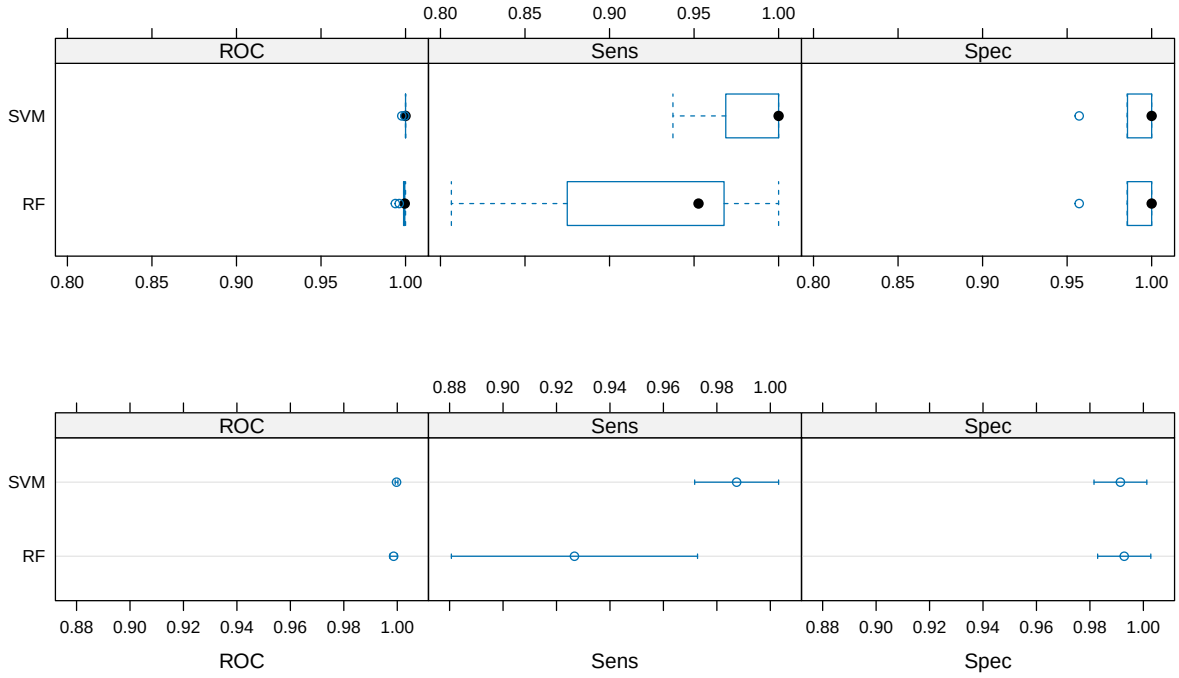
Comparación de modelos

```
model_list <- list(SVM = svm, RF = rf2)
res <- resamples(model_list)
summary(res)
```

```
##
## Call:
## summary.resamples(object = res)
##
## Models: SVM, RF
## Number of resamples: 10
##
## ROC
##          Min.   1st Qu.    Median      Mean  3rd Qu. Max. NA's
## SVM 0.9977679 1.0000000 1.0000000 0.9997307 1.000000    1    0
## RF  0.9939732 0.9989095 0.9995392 0.9986354 0.999552    1    0
##
## Sens
##          Min.   1st Qu.   Median      Mean   3rd Qu. Max. NA's
```

```
## SVM 0.9375000 0.9765625 1.000000 0.9873992 1.0000000    1    0
## RF  0.8064516 0.8906250 0.952621 0.9267137 0.9677419    1    0
##
## Spec
##          Min.   1st Qu. Median      Mean 3rd Qu. Max. NA's
## SVM 0.9571429 0.9857143      1 0.9914079       1    1    0
## RF  0.9571429 0.9892857      1 0.9928364       1    1    0
```

```
p1 = bwplot(res)
p2 = dotplot(res)
grid.arrange(p1, p2)
```



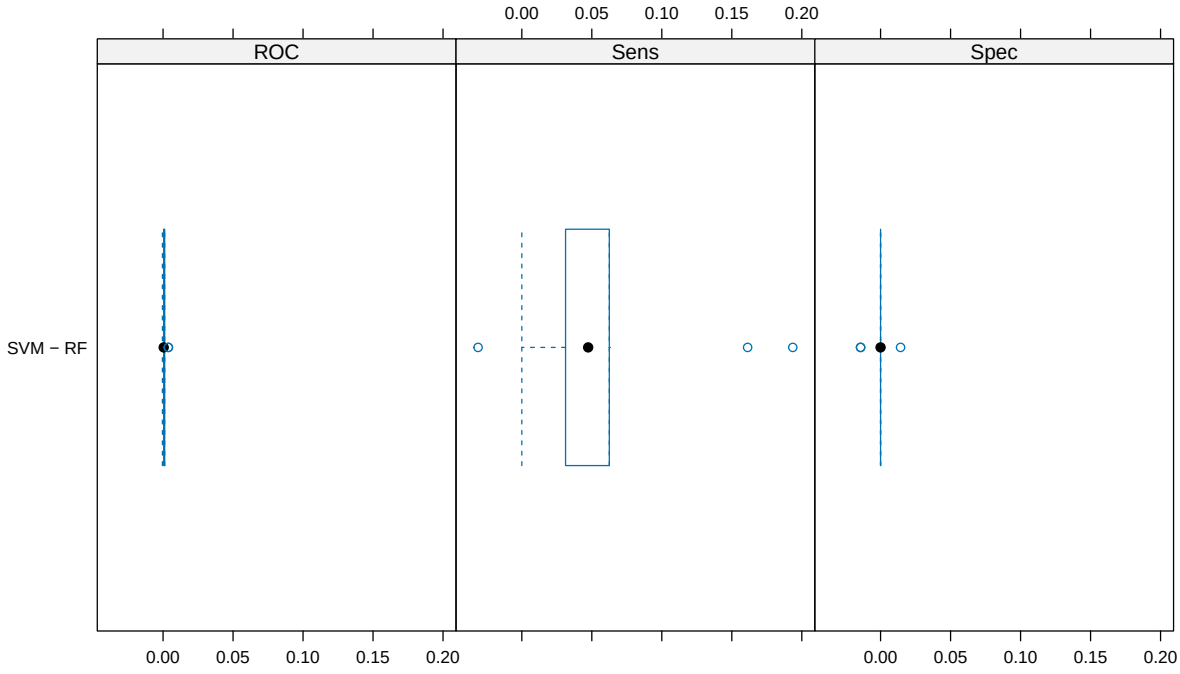
Confidence Level: 0.95

```
difValues <- diff(res)
summary(difValues)
```

```
##
## Call:
## summary.diff.resamples(object = difValues)
##
## p-value adjustment: bonferroni
## Upper diagonal: estimates of the difference
## Lower diagonal: p-value for H0: difference = 0
##
## ROC
##     SVM      RF
## SVM          0.001095
## RF  0.04343
##
```

```
## Sens
##     SVM     RF
## SVM         0.06069
## RF  0.02085
##
## Spec
##     SVM    RF
## SVM        -0.001429
## RF  0.5911
```

```
bwplot(difValues, layout = c(3, 1))
```



Evaluación en test set: Random Forest

```
prediction = predict(rf2, newdata = test_set)
postResample(prediction, test_set$type)
```

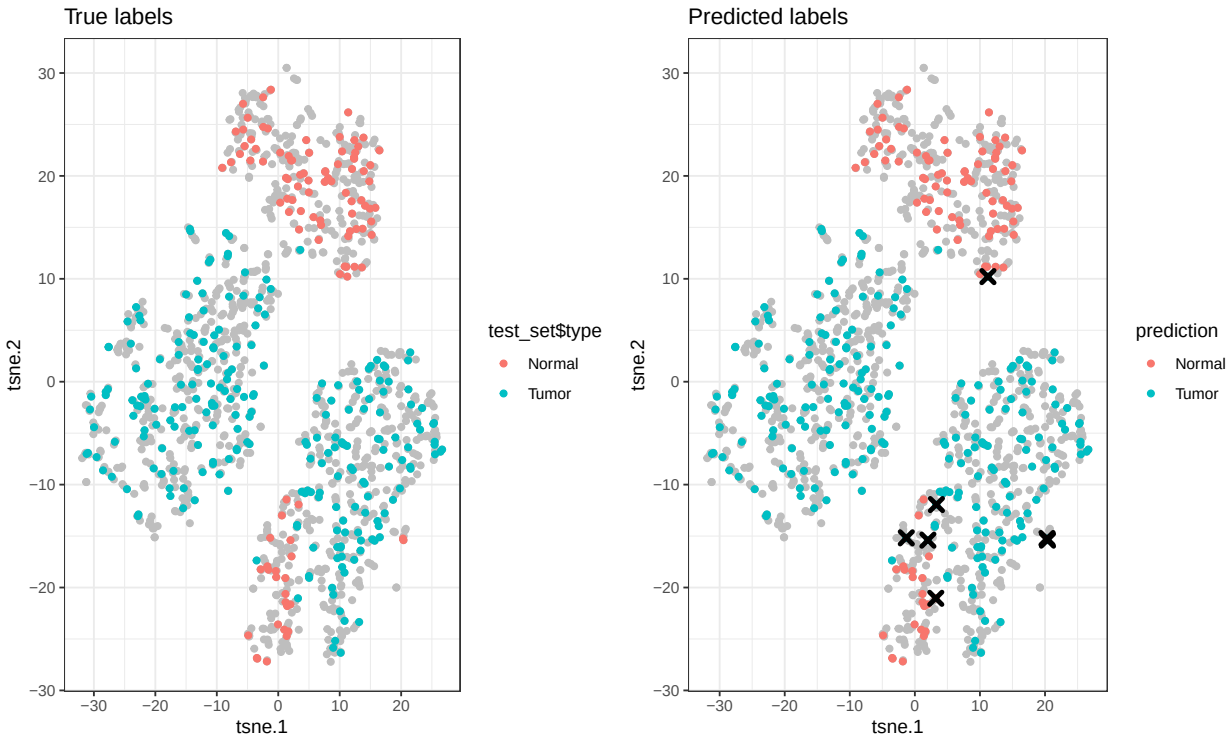
```
##  Accuracy     Kappa
## 0.9791667 0.9506048
```

```
confusionMatrix(prediction, test_set$type)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Normal Tumor
```

```
##     Normal     98     1
##     Tumor       6   231
##
##                Accuracy : 0.9792
##                  95% CI : (0.9575, 0.9916)
##     No Information Rate : 0.6905
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.9506
##
##  Mcnemar's Test P-Value : 0.1306
##
##             Sensitivity : 0.9423
##             Specificity : 0.9957
##          Pos Pred Value : 0.9899
##          Neg Pred Value : 0.9747
##              Prevalence : 0.3095
##          Detection Rate : 0.2917
##    Detection Prevalence : 0.2946
##       Balanced Accuracy : 0.9690
##
##        'Positive' Class : Normal
##
```

```
p1 = ggplot() +
  geom_point(data = tsne.df, mapping = aes(x = tsne.1, y = tsne.2), color = 'gray') +
  geom_point(data = tsne.df[rownames(test_set),], mapping = aes(x = tsne.1, y = tsne.2, color = test_se
  ggtitle(label = 'True labels') + theme_bw()

tsne_test = tsne.df[rownames(test_set),]
tsne_test$prediction <- prediction

p2 = ggplot() +
  geom_point(data = tsne.df, mapping = aes(x = tsne.1, y = tsne.2), color = 'gray') +
  geom_point(data = tsne_test, mapping = aes(x = tsne.1, y = tsne.2, color = prediction)) +
  geom_point(data = subset(tsne_test, prediction != type), mapping = aes(x = tsne.1, y = tsne.2), shape
  ggtitle(label = 'Predicted labels') + theme_bw()

grid.arrange(p1, p2, ncol = 2)
```



Evaluación en test set: SVM

```
prediction = predict(svm, newdata = test_set)
postResample(prediction, test_set$type)
```

```
##  Accuracy     Kappa
## 0.9940476 0.9861478
```

```
confusionMatrix(prediction, test_set$type)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Normal Tumor
##     Normal    104     2
##     Tumor       0   230
##
##                Accuracy : 0.994
##                  95% CI : (0.9787, 0.9993)
##     No Information Rate : 0.6905
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.9861
##
##  Mcnemar's Test P-Value : 0.4795
##
```

```
##             Sensitivity : 1.0000
##             Specificity : 0.9914
##          Pos Pred Value : 0.9811
##          Neg Pred Value : 1.0000
##              Prevalence : 0.3095
##          Detection Rate : 0.3095
##    Detection Prevalence : 0.3155
##       Balanced Accuracy : 0.9957
##
##        'Positive' Class : Normal
##
```

Genes más importantes (varImp): Tumor vs Normal

La función varImp muestra la importancia de cada gen. Las columnas Normal y Tumor indican la dirección: ↑ Normal significa que el gen contribuye a clasificar como Normal.

```
roc_imp_rf <- varImp(rf2, scale = FALSE)
rownames(roc_imp_rf$importance) = sapply(strsplit(rownames(roc_imp_rf$importance), '_'), `[`, 2)
rownames(roc_imp_rf$importance) = cancer_genes$symbol[match(rownames(roc_imp_rf$importance), cancer_gene

top_idx = order(roc_imp_rf$importance$Normal, decreasing = TRUE)[1:20]
top_genes_rf = data.frame(
  Gen = rownames(roc_imp_rf$importance)[top_idx],
  Importancia_Normal = round(roc_imp_rf$importance$Normal[top_idx], 4),
  Importancia_Tumor = round(roc_imp_rf$importance$Tumor[top_idx], 4),
  Direccion = ifelse(roc_imp_rf$importance$Normal[top_idx] > roc_imp_rf$importance$Tumor[top_idx],
                     "↑ Normal", "↑ Tumor")
)
knitr::kable(top_genes_rf, caption = "RF: Top 20 genes con dirección")
```

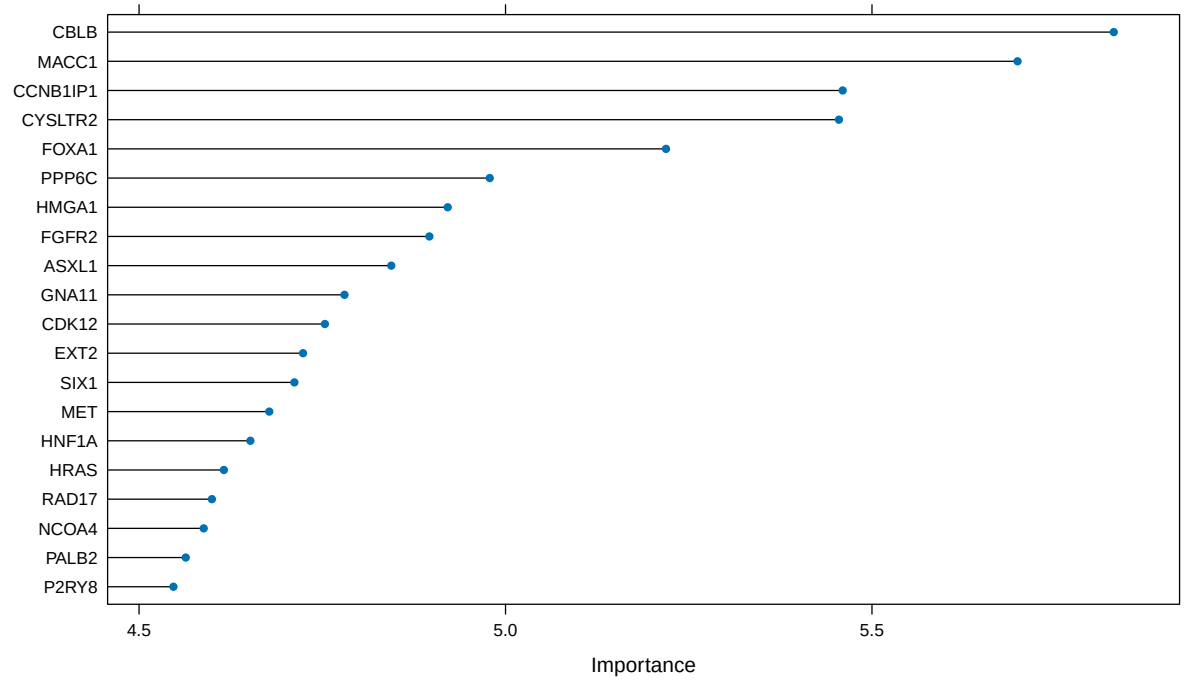
Random Forest

Table 1: RF: Top 20 genes con dirección

Gen	Importancia_Normal	Importancia_Tumor	Direccion
CBLB	5.8299	5.8299	↑ Tumor
MACC1	5.6986	5.6986	↑ Tumor
CCNB1IP1	5.4600	5.4600	↑ Tumor
CYSLTR2	5.4549	5.4549	↑ Tumor
FOXA1	5.2190	5.2190	↑ Tumor
PPP6C	4.9784	4.9784	↑ Tumor
HMGA1	4.9211	4.9211	↑ Tumor
FGFR2	4.8961	4.8961	↑ Tumor
ASXL1	4.8441	4.8441	↑ Tumor
GNA11	4.7803	4.7803	↑ Tumor
CDK12	4.7536	4.7536	↑ Tumor
EXT2	4.7237	4.7237	↑ Tumor
SIX1	4.7119	4.7119	↑ Tumor
MET	4.6777	4.6777	↑ Tumor

Gen	Importancia_Normal	Importancia_Tumor	Direccion
HNF1A	4.6519	4.6519	↑ Tumor
HRAS	4.6157	4.6157	↑ Tumor
RAD17	4.5994	4.5994	↑ Tumor
NCOA4	4.5882	4.5882	↑ Tumor
PALB2	4.5637	4.5637	↑ Tumor
P2RY8	4.5469	4.5469	↑ Tumor

```
plot(roc_imp_rf, top = 20)
```



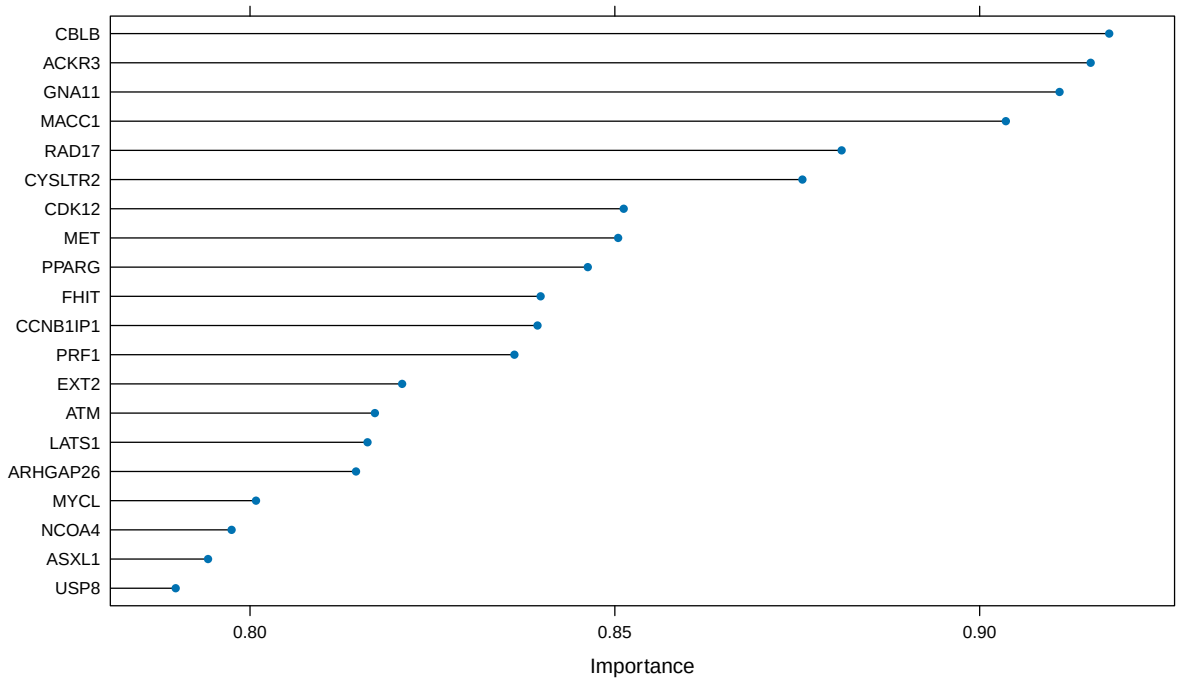

```
roc_imp_svm <- varImp(svm, scale = FALSE)
rownames(roc_imp_svm$importance) = sapply(strsplit(rownames(roc_imp_svm$importance), '_'), `[`, 2)
rownames(roc_imp_svm$importance) = cancer_genes$symbol[match(rownames(roc_imp_svm$importance), cancer_g

top_idx = order(roc_imp_svm$importance$Normal, decreasing = TRUE)[1:20]
top_genes_svm = data.frame(
  Gen = rownames(roc_imp_svm$importance)[top_idx],
  Importancia_Normal = round(roc_imp_svm$importance$Normal[top_idx], 4),
  Importancia_Tumor = round(roc_imp_svm$importance$Tumor[top_idx], 4),
  Direccion = ifelse(roc_imp_svm$importance$Normal[top_idx] > roc_imp_svm$importance$Tumor[top_idx],
                     "↑ Normal", "↑ Tumor")
)
knitr::kable(top_genes_svm, caption = "SVM: Top 20 genes con dirección")
```

SVM

Table 2: SVM: Top 20 genes con dirección

Gen	Importancia_Normal	Importancia_Tumor	Direccion
CBLB	0.9178	0.9178	↑ Tumor
ACKR3	0.9152	0.9152	↑ Tumor
GNA11	0.9109	0.9109	↑ Tumor
MACC1	0.9036	0.9036	↑ Tumor
RAD17	0.8811	0.8811	↑ Tumor
CYSLTR2	0.8757	0.8757	↑ Tumor
CDK12	0.8512	0.8512	↑ Tumor
MET	0.8505	0.8505	↑ Tumor
PPARG	0.8463	0.8463	↑ Tumor
FHIT	0.8398	0.8398	↑ Tumor
CCNB1IP1	0.8394	0.8394	↑ Tumor
PRF1	0.8362	0.8362	↑ Tumor
EXT2	0.8208	0.8208	↑ Tumor
ATM	0.8171	0.8171	↑ Tumor
LATS1	0.8161	0.8161	↑ Tumor
ARHGAP26	0.8145	0.8145	↑ Tumor
MYCL	0.8008	0.8008	↑ Tumor
NCOA4	0.7975	0.7975	↑ Tumor
ASXL1	0.7942	0.7942	↑ Tumor
USP8	0.7898	0.7898	↑ Tumor

```
plot(roc_imp_svm, top = 20)
```



```r
top_20_rf_TN = rownames(roc_imp_rf$importance[order(roc_imp_rf$importance$Normal, decreasing = T),])[1:
top_20_svm_TN = rownames(roc_imp_svm$importance[order(roc_imp_svm$importance$Normal, decreasing = T),])
comunes = top_20_rf_TN[top_20_rf_TN %in% top_20_svm_TN]
cat("Genes comunes (", length(comunes), "):", paste(comunes, collapse = ", "))
```

Genes comunes RF y SVM

```
## Genes comunes ( 11 ): CBLB, MACC1, CCNB1IP1, CYSLTR2, ASXL1, GNA11, CDK12, EXT2, MET, RAD17, NCOA4
```

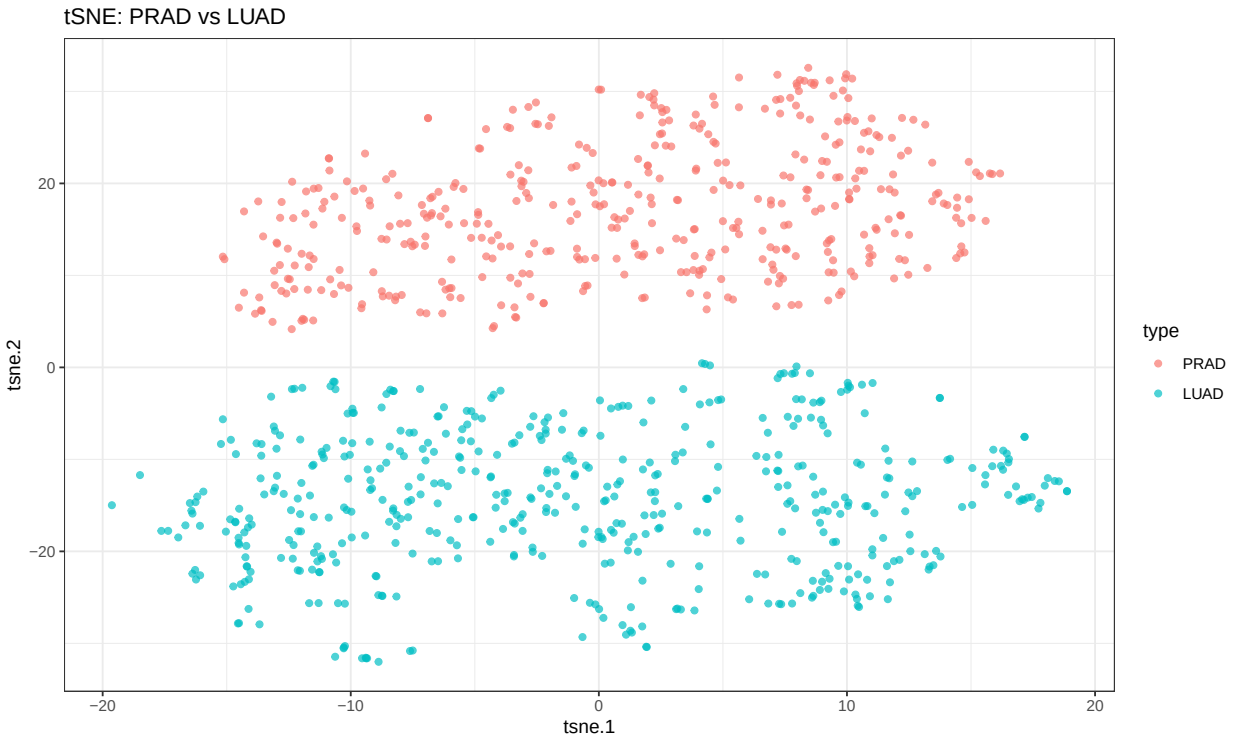
Clasificación PRAD vs LUAD

Construcción de matrices

```r
train_set = tumor_train_set
colnames(train_set) = paste0('g_', colnames(train_set)[1:(ncol(train_set))])
train_set$type = factor(c(rep('PRAD', (ncol(tumor_A_train)-2)), rep('LUAD', (ncol(tumor_B_train)-2))), l

test_set = tumor_test_set
colnames(test_set) = paste0('g_', colnames(test_set)[1:(ncol(test_set))])
test_set$type = factor(c(rep('PRAD', (ncol(tumor_A_test)-2)), rep('LUAD', (ncol(tumor_B_test)-2))), leve

tmp = rbind(train_set, test_set)
all_data = tmp[, 1:(ncol(tmp)-1)]
all_data = predict(preProcess(all_data, method = c("center", "scale", "YeoJohnson", "nzv")), all_data)
all_data$type = tmp$type
train_set = all_data[1:nrow(train_set),]
test_set = all_data[(nrow(train_set)+1):(nrow(train_set)+nrow(test_set)),]
```

tSNE

```r
set.seed(900808)
tsne <- Rtsne(all_data[, colnames(all_data) != 'type'], dims = 2, perplexity = 15, verbose = FALSE, max_
tsne.df = as.data.frame(tsne$Y)
rownames(tsne.df) = rownames(all_data)
colnames(tsne.df) = c('tsne.1', 'tsne.2')
tsne.df$type = all_data$type

ggplot(tsne.df, aes(x = tsne.1, y = tsne.2, color = type)) +
  geom_point(alpha = 0.7) +
  ggtitle("tSNE: PRAD vs LUAD") +
  theme_bw()
```



Entrenamiento y evaluación

```
sel = colnames(train_set[1:(ncol(train_set)-1)])
formula <- as.formula(paste("type ~ ", paste(sel, collapse = "+")))
to_forest = train_set
train_control <- trainControl(method = "cv", number = 10, classProbs = TRUE, summaryFunction = twoClass

set.seed(900808)
rf2 <- train(formula, data = to_forest, method = "rf", tuneLength = 10, importance = T, trControl = tra
rf2
```

```
## Random Forest
##
## 697 samples
## 506 predictors
##   2 classes: 'PRAD', 'LUAD'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 628, 628, 627, 627, 627, 627, ...
## Resampling results across tuning parameters:
##
##   mtry  ROC  Sens  Spec
##     2   1    1     1
##     3   1    1     1
##     6   1    1     1
##    12   1    1     1
```

```
##    23   1    1     1
##    43   1    1     1
##    80   1    1     1
##   147   1    1     1
##   273   1    1     1
##   506   1    1     1
##
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 2.
```

```
postResample(predict(rf2, newdata = to_forest), to_forest$type)
```

```
## Accuracy    Kappa
##        1        1
```

```
to_svm = train_set
set.seed(900808)
svm <- train(formula, data = to_svm, method = "svmRadial", tuneLength = 10, importance = T, trControl =
svm
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 697 samples
## 506 predictors
##   2 classes: 'PRAD', 'LUAD'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 628, 628, 627, 627, 627, 627, ...
## Resampling results across tuning parameters:
##
##   C       ROC  Sens  Spec
##     0.25  1    1     1
##     0.50  1    1     1
##     1.00  1    1     1
##     2.00  1    1     1
##     4.00  1    1     1
##     8.00  1    1     1
##    16.00  1    1     1
##    32.00  1    1     1
##    64.00  1    1     1
##   128.00  1    1     1
##
## Tuning parameter 'sigma' was held constant at a value of 0.00215776
## ROC was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.00215776 and C = 0.25.
```

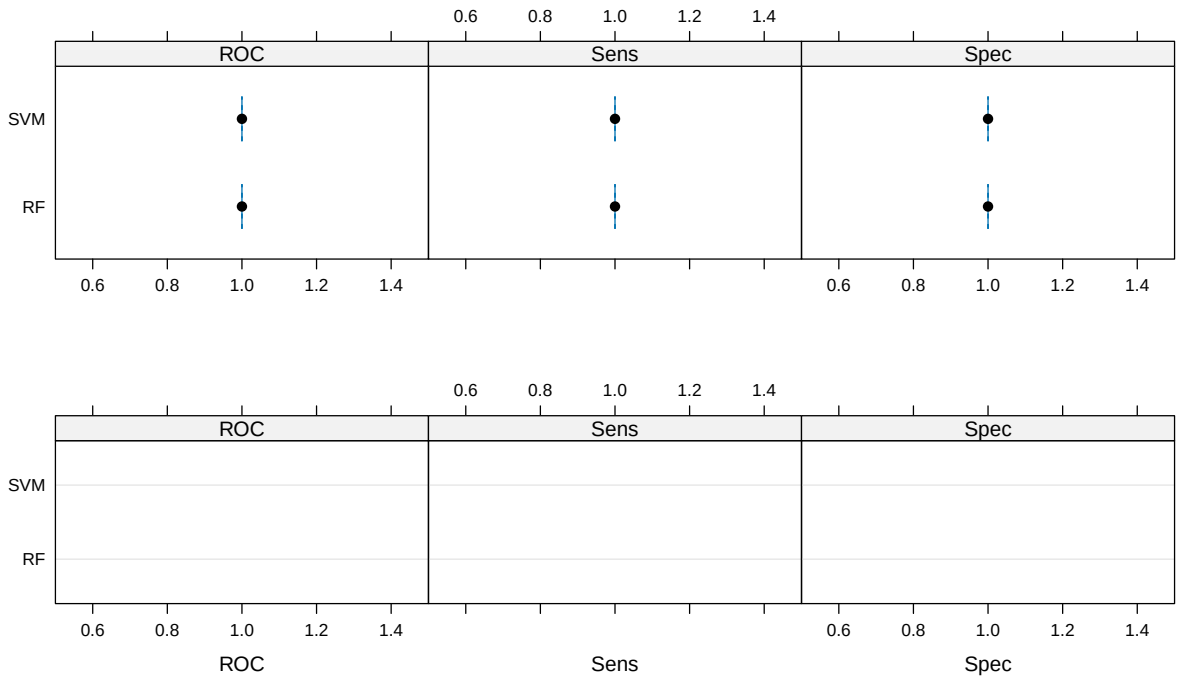
```
postResample(predict(svm, newdata = to_svm), to_svm$type)
```

```
## Accuracy    Kappa
##        1        1
```

```
model_list <- list(SVM = svm, RF = rf2)
res <- resamples(model_list)
summary(res)
```

```
##
## Call:
## summary.resamples(object = res)
##
## Models: SVM, RF
## Number of resamples: 10
##
## ROC
##     Min. 1st Qu. Median Mean 3rd Qu. Max. NA's
## SVM    1       1      1    1       1    1    0
## RF     1       1      1    1       1    1    0
##
## Sens
##     Min. 1st Qu. Median Mean 3rd Qu. Max. NA's
## SVM    1       1      1    1       1    1    0
## RF     1       1      1    1       1    1    0
##
## Spec
##     Min. 1st Qu. Median Mean 3rd Qu. Max. NA's
## SVM    1       1      1    1       1    1    0
## RF     1       1      1    1       1    1    0
```

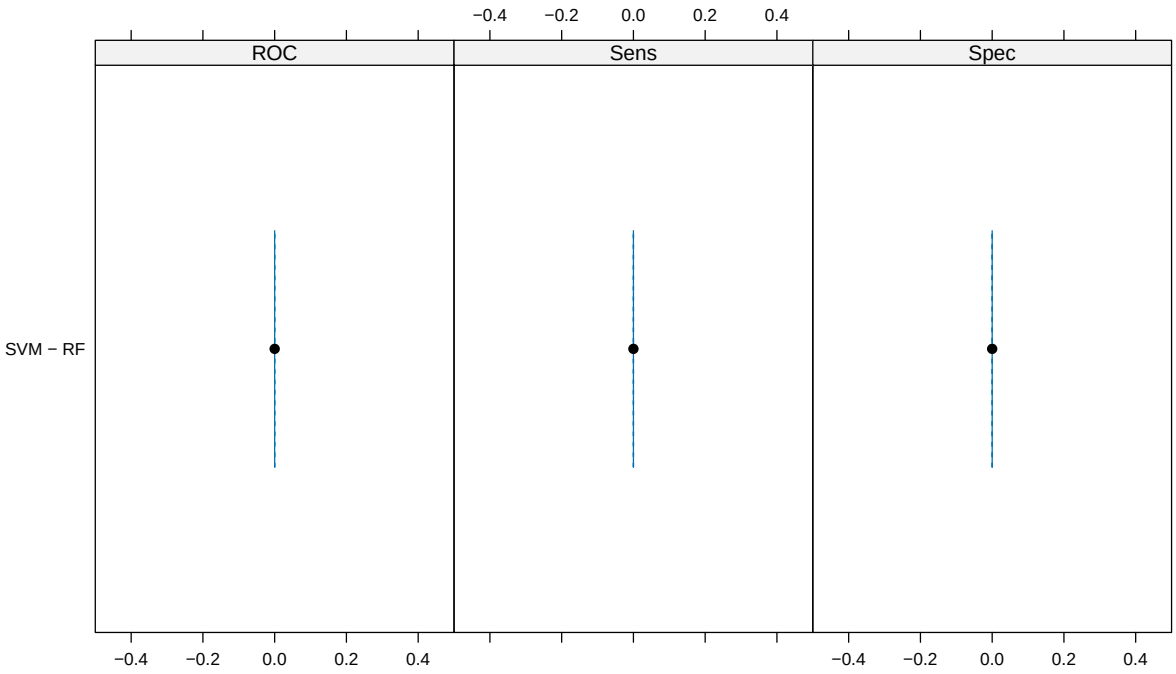
```
p1 = bwplot(res)
p2 = dotplot(res)
grid.arrange(p1, p2)
```



```r
difValues <- diff(res)
summary(difValues)
```

```
##
## Call:
## summary.diff.resamples(object = difValues)
##
## p-value adjustment: bonferroni
## Upper diagonal: estimates of the difference
## Lower diagonal: p-value for H0: difference = 0
##
## ROC
##     SVM RF
## SVM      0
## RF  NA
##
## Sens
##     SVM RF
## SVM      0
## RF  NA
##
## Spec
##     SVM RF
## SVM      0
## RF  NA
```

```r
bwplot(difValues, layout = c(3, 1))
```



```r
cat("=== Random Forest: Test ===\n")
```

```
## === Random Forest: Test ===
```

```r
prediction = predict(rf2, newdata = test_set)
postResample(prediction, test_set$type)
```

```
## Accuracy    Kappa
##        1        1
```

```r
confusionMatrix(prediction, test_set$type)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction PRAD LUAD
##       PRAD  106    0
##       LUAD    0  126
##
##                Accuracy : 1
##                  95% CI : (0.9842, 1)
##     No Information Rate : 0.5431
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.4569
##          Detection Rate : 0.4569
##    Detection Prevalence : 0.4569
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : PRAD
##
```

```r
cat("\n=== SVM: Test ===\n")
```

```
##
## === SVM: Test ===
```

```r
prediction = predict(svm, newdata = test_set)
postResample(prediction, test_set$type)
```

```
## Accuracy    Kappa
##        1        1
```

```r
confusionMatrix(prediction, test_set$type)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction PRAD LUAD
##       PRAD  106    0
##       LUAD    0  126
##
##                Accuracy : 1
##                  95% CI : (0.9842, 1)
##     No Information Rate : 0.5431
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.0000
##             Specificity : 1.0000
##          Pos Pred Value : 1.0000
##          Neg Pred Value : 1.0000
##              Prevalence : 0.4569
##          Detection Rate : 0.4569
##    Detection Prevalence : 0.4569
##       Balanced Accuracy : 1.0000
##
##        'Positive' Class : PRAD
##
```

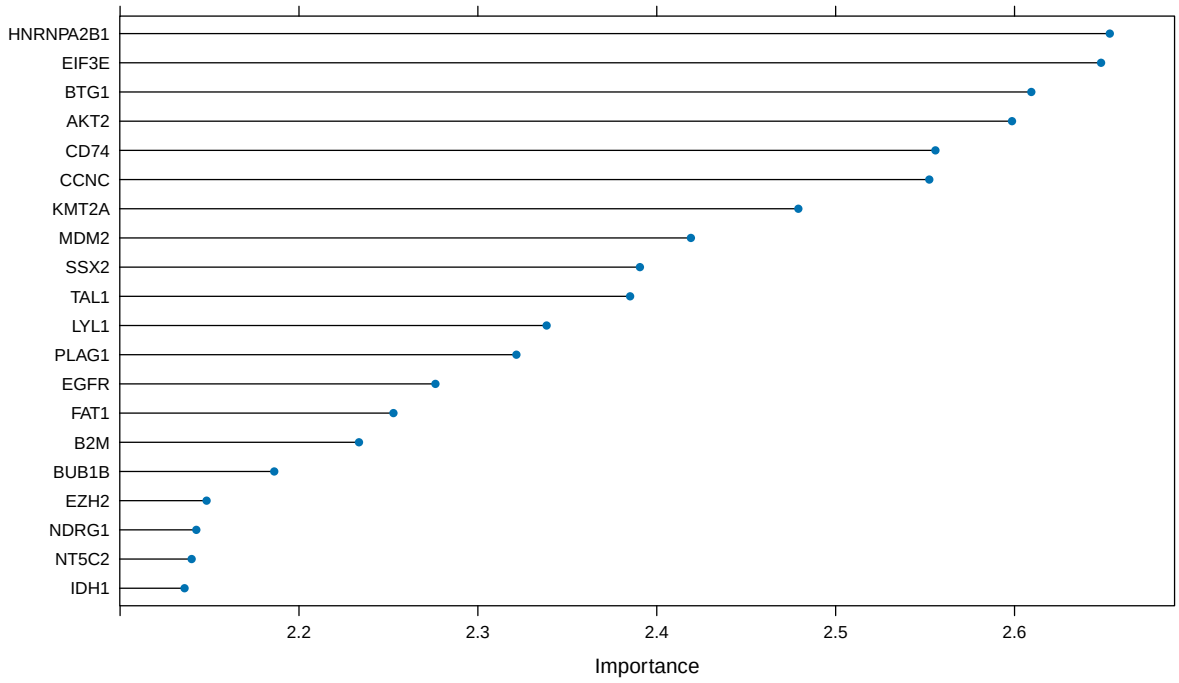
Genes importantes PRAD vs LUAD

```r
roc_imp_rf <- varImp(rf2, scale = FALSE)
rownames(roc_imp_rf$importance) = sapply(strsplit(rownames(roc_imp_rf$importance), '_'), `[`, 2)
rownames(roc_imp_rf$importance) = cancer_genes$symbol[match(rownames(roc_imp_rf$importance), cancer_gene

top_idx = order(roc_imp_rf$importance$PRAD, decreasing = TRUE)[1:20]
top_genes_rf_TT = data.frame(
  Gen = rownames(roc_imp_rf$importance)[top_idx],
  Importancia_PRAD = round(roc_imp_rf$importance$PRAD[top_idx], 4),
  Importancia_LUAD = round(roc_imp_rf$importance$LUAD[top_idx], 4),
  Direccion = ifelse(roc_imp_rf$importance$PRAD[top_idx] > roc_imp_rf$importance$LUAD[top_idx],
                     "↑ PRAD", "↑ LUAD")
)
knitr::kable(top_genes_rf_TT, caption = "RF: Top 20 genes PRAD vs LUAD con dirección")
```

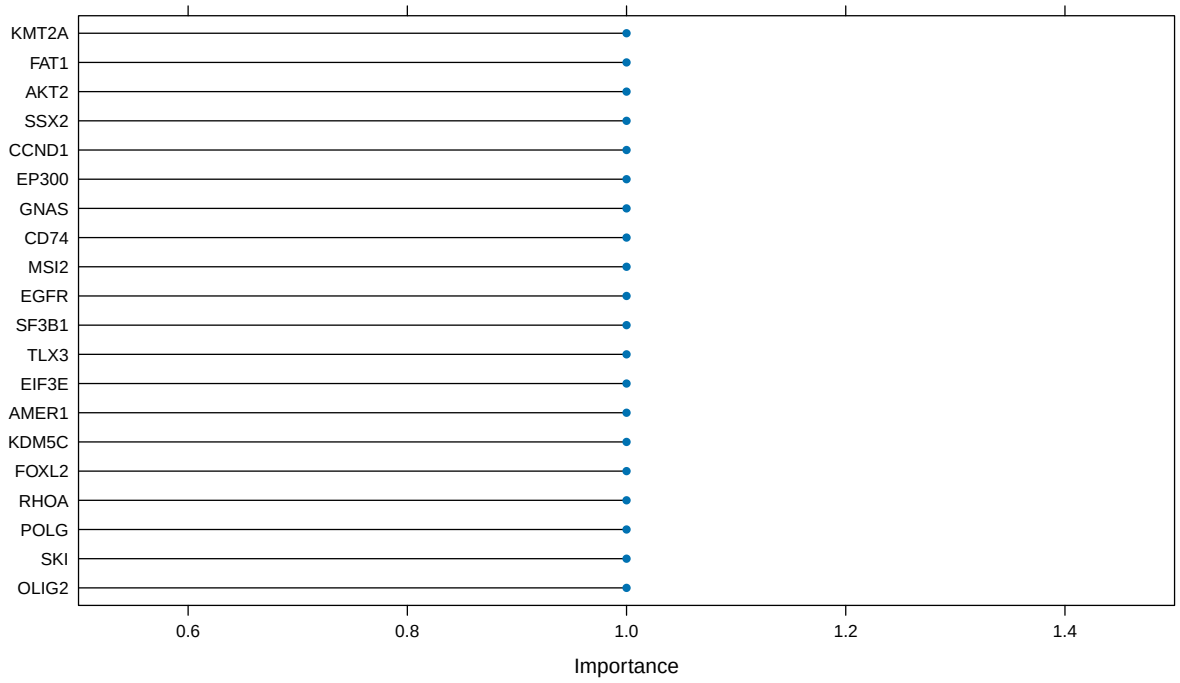
Table 3: RF: Top 20 genes PRAD vs LUAD con dirección

Gen	Importancia_PRAD	Importancia_LUAD	Direccion
HNRNPA2B1	2.6532	2.6532	↑ LUAD
EIF3E	2.6484	2.6484	↑ LUAD
BTG1	2.6094	2.6094	↑ LUAD
AKT2	2.5986	2.5986	↑ LUAD
CD74	2.5558	2.5558	↑ LUAD
CCNC	2.5523	2.5523	↑ LUAD
KMT2A	2.4791	2.4791	↑ LUAD
MDM2	2.4191	2.4191	↑ LUAD
SSX2	2.3906	2.3906	↑ LUAD
TAL1	2.3851	2.3851	↑ LUAD
LYL1	2.3384	2.3384	↑ LUAD
PLAG1	2.3215	2.3215	↑ LUAD
EGFR	2.2763	2.2763	↑ LUAD
FAT1	2.2528	2.2528	↑ LUAD
B2M	2.2335	2.2335	↑ LUAD
BUB1B	2.1862	2.1862	↑ LUAD
EZH2	2.1483	2.1483	↑ LUAD
NDRG1	2.1426	2.1426	↑ LUAD
NT5C2	2.1400	2.1400	↑ LUAD
IDH1	2.1360	2.1360	↑ LUAD

```
plot(roc_imp_rf, top = 20)
```



```r
top_20_rf_TT = rownames(roc_imp_rf$importance[order(roc_imp_rf$importance$PRAD, decreasing = T),])[1:20]

roc_imp_svm <- varImp(svm, scale = FALSE)
rownames(roc_imp_svm$importance) = sapply(strsplit(rownames(roc_imp_svm$importance), '_'), `[`, 2)
rownames(roc_imp_svm$importance) = cancer_genes$symbol[match(rownames(roc_imp_svm$importance), cancer_g
top_20_svm_TT = rownames(roc_imp_svm$importance[order(roc_imp_svm$importance$PRAD, decreasing = T),])[1
plot(roc_imp_svm, top = 20)
```




```r
comunes = top_20_rf_TT[top_20_rf_TT %in% top_20_svm_TT]
cat("Genes comunes (", length(comunes), "):", paste(comunes, collapse = ", "))
```

```
## Genes comunes ( 15 ): HNRNPA2B1, EIF3E, BTG1, AKT2, CD74, CCNC, KMT2A, SSX2, EGFR, FAT1, B2M, EZH2,
```

Fase 2: Solo oncogenes

En esta fase repetimos el análisis Tumor vs Normal usando exclusivamente los oncogenes, eliminando los TSGs. La única diferencia en el código es filtrar con `oncogenes_only$entrez` en vez de `cancer_genes$entrez`.

```r
normal_A_f2 = read.table('sources/prostate-rsem-fpkm-gtex.txt.gz', header = T)
normal_B_f2 = read.table('sources/lung-rsem-fpkm-gtex.txt.gz', header = T)
tumor_A_f2 = read.table('sources/prad-rsem-fpkm-tcga-t.txt.gz', header = T)
tumor_B_f2 = read.table('sources/luad-rsem-fpkm-tcga-t.txt.gz', header = T)

oncogenes_only = subset(cancer_genes, type == "oncogene")
cat("Oncogenes:", nrow(oncogenes_only), "\n")
```

```
## Oncogenes: 256
```

```
tumor_A_f2 = subset(tumor_A_f2, Entrez_Gene_Id %in% oncogenes_only$entrez)
rep = names(table(tumor_A_f2$Entrez_Gene_Id)[which(table(tumor_A_f2$Entrez_Gene_Id)>1)])
tumor_A_f2 = subset(tumor_A_f2, !Entrez_Gene_Id %in% rep | Hugo_Symbol %in% subset(oncogenes_only, entre
rownames(tumor_A_f2) = tumor_A_f2$Entrez_Gene_Id

tumor_B_f2 = subset(tumor_B_f2, Entrez_Gene_Id %in% oncogenes_only$entrez)
rep = names(table(tumor_B_f2$Entrez_Gene_Id)[which(table(tumor_B_f2$Entrez_Gene_Id)>1)])
tumor_B_f2 = subset(tumor_B_f2, !Entrez_Gene_Id %in% rep | Hugo_Symbol %in% subset(oncogenes_only, entre
rownames(tumor_B_f2) = tumor_B_f2$Entrez_Gene_Id

normal_A_f2 = subset(normal_A_f2, Entrez_Gene_Id %in% oncogenes_only$entrez)
rep = names(table(normal_A_f2$Entrez_Gene_Id)[which(table(normal_A_f2$Entrez_Gene_Id)>1)])
normal_A_f2 = subset(normal_A_f2, !Entrez_Gene_Id %in% rep | Hugo_Symbol %in% subset(oncogenes_only, en
rownames(normal_A_f2) = normal_A_f2$Entrez_Gene_Id

normal_B_f2 = subset(normal_B_f2, Entrez_Gene_Id %in% oncogenes_only$entrez)
rep = names(table(normal_B_f2$Entrez_Gene_Id)[which(table(normal_B_f2$Entrez_Gene_Id)>1)])
normal_B_f2 = subset(normal_B_f2, !Entrez_Gene_Id %in% rep | Hugo_Symbol %in% subset(oncogenes_only, en
rownames(normal_B_f2) = normal_B_f2$Entrez_Gene_Id

cat("Genes tras filtro:", nrow(tumor_A_f2), "\n")
```

```
## Genes tras filtro: 253
```

```
set.seed(900808)
tumor_A_train_f2 = tumor_A_f2[,c("Hugo_Symbol","Entrez_Gene_Id", sample(x = colnames(tumor_A_f2)[3:ncol
tumor_A_test_f2 = cbind(tumor_A_f2[,1:2], tumor_A_f2[,!colnames(tumor_A_f2) %in% colnames(tumor_A_train
tumor_B_train_f2 = tumor_B_f2[,c("Hugo_Symbol","Entrez_Gene_Id", sample(x = colnames(tumor_B_f2)[3:ncol
tumor_B_test_f2 = cbind(tumor_B_f2[,1:2], tumor_B_f2[,!colnames(tumor_B_f2) %in% colnames(tumor_B_train
normal_A_train_f2 = normal_A_f2[,c("Hugo_Symbol","Entrez_Gene_Id", sample(x = colnames(normal_A_f2)[3:n
normal_A_test_f2 = cbind(normal_A_f2[,1:2], normal_A_f2[,!colnames(normal_A_f2) %in% colnames(normal_A_
normal_B_train_f2 = normal_B_f2[,c("Hugo_Symbol","Entrez_Gene_Id", sample(x = colnames(normal_B_f2)[3:n
normal_B_test_f2 = cbind(normal_B_f2[,1:2], normal_B_f2[,!colnames(normal_B_f2) %in% colnames(normal_B_

tumor_A_train_f2 = subset(tumor_A_train_f2, Entrez_Gene_Id %in% tumor_B_train_f2$Entrez_Gene_Id)
tumor_B_train_f2 = subset(tumor_B_train_f2, Entrez_Gene_Id %in% tumor_A_train_f2$Entrez_Gene_Id)
tumor_train_set_f2 = as.data.frame(t(cbind(tumor_A_train_f2[,3:ncol(tumor_A_train_f2)], tumor_B_train_f
normal_A_train_f2 = subset(normal_A_train_f2, Entrez_Gene_Id %in% normal_B_train_f2$Entrez_Gene_Id)
normal_B_train_f2 = subset(normal_B_train_f2, Entrez_Gene_Id %in% normal_A_train_f2$Entrez_Gene_Id)
normal_train_set_f2 = as.data.frame(t(cbind(normal_A_train_f2[,3:ncol(normal_A_train_f2)], normal_B_tra
train_set_f2 = rbind(tumor_train_set_f2, normal_train_set_f2)
colnames(train_set_f2) = paste0('g_', colnames(train_set_f2)[1:(ncol(train_set_f2))])
train_set_f2$type = factor(c(rep('Tumor', nrow(tumor_train_set_f2)), rep('Normal', nrow(normal_train_se

tumor_A_test_f2 = subset(tumor_A_test_f2, Entrez_Gene_Id %in% tumor_B_test_f2$Entrez_Gene_Id)
tumor_B_test_f2 = subset(tumor_B_test_f2, Entrez_Gene_Id %in% tumor_A_test_f2$Entrez_Gene_Id)
tumor_test_set_f2 = as.data.frame(t(cbind(tumor_A_test_f2[,3:ncol(tumor_A_test_f2)], tumor_B_test_f2[,3
normal_A_test_f2 = subset(normal_A_test_f2, Entrez_Gene_Id %in% normal_B_test_f2$Entrez_Gene_Id)
normal_B_test_f2 = subset(normal_B_test_f2, Entrez_Gene_Id %in% normal_A_test_f2$Entrez_Gene_Id)
normal_test_set_f2 = as.data.frame(t(cbind(normal_A_test_f2[,3:ncol(normal_A_test_f2)], normal_B_test_f
test_set_f2 = rbind(tumor_test_set_f2, normal_test_set_f2)
```

```r
colnames(test_set_f2) = paste0('g_', colnames(test_set_f2)[1:(ncol(test_set_f2))])
test_set_f2$type = factor(c(rep('Tumor', nrow(tumor_test_set_f2)), rep('Normal', nrow(normal_test_set_f2

tmp = rbind(train_set_f2, test_set_f2)
all_data_f2 = tmp[, 1:(ncol(tmp)-1)]
all_data_f2 = predict(preProcess(all_data_f2, method = c("center", "scale", "YeoJohnson", "nzv")), all_d
all_data_f2$type = tmp$type
train_set_f2 = all_data_f2[1:nrow(train_set_f2),]
test_set_f2 = all_data_f2[(nrow(train_set_f2)+1):(nrow(train_set_f2)+nrow(test_set_f2)),]

sel = colnames(train_set_f2[1:(ncol(train_set_f2)-1)])
formula <- as.formula(paste("type ~ ", paste(sel, collapse = "+")))
train_control <- trainControl(method = "cv", number = 10, classProbs = TRUE, summaryFunction = twoClass

set.seed(900808)
rf2_onco <- train(formula, data = train_set_f2, method = "rf", tuneLength = 10, importance = T, trContro

set.seed(900808)
svm_onco <- train(formula, data = train_set_f2, method = "svmRadial", tuneLength = 10, importance = T,
```

Comparación Fase 1 vs Fase 2

```r
res_f1_rf = postResample(predict(rf_TN, newdata = test_set), test_set$type)
res_f1_svm = postResample(predict(svm_TN, newdata = test_set), test_set$type)
res_f2_rf = postResample(predict(rf2_onco, newdata = test_set_f2), test_set_f2$type)
res_f2_svm = postResample(predict(svm_onco, newdata = test_set_f2), test_set_f2$type)

comparacion = data.frame(
  Modelo = c("Random Forest", "SVM Radial"),
  Fase1_506genes = c(res_f1_rf["Accuracy"], res_f1_svm["Accuracy"]),
  Fase2_253oncogenes = c(res_f2_rf["Accuracy"], res_f2_svm["Accuracy"]),
  Diferencia = c(res_f2_rf["Accuracy"] - res_f1_rf["Accuracy"], res_f2_svm["Accuracy"] - res_f1_svm["Ac
)
knitr::kable(comparacion, digits = 4, caption = "Comparación: 506 genes (Fase 1) vs 253 oncogenes (Fase
```

Table 4: Comparación: 506 genes (Fase 1) vs 253 oncogenes (Fase 2)

Modelo	Fase1_506genes	Fase2_253oncogenes	Diferencia
Random Forest	NA	0.9851	NA
SVM Radial	NA	0.9970	NA

Conclusión: Con la mitad de genes (solo oncogenes), el rendimiento se mantiene o mejora ligeramente. Esto justifica centrar el análisis de explicabilidad exclusivamente en oncogenes.

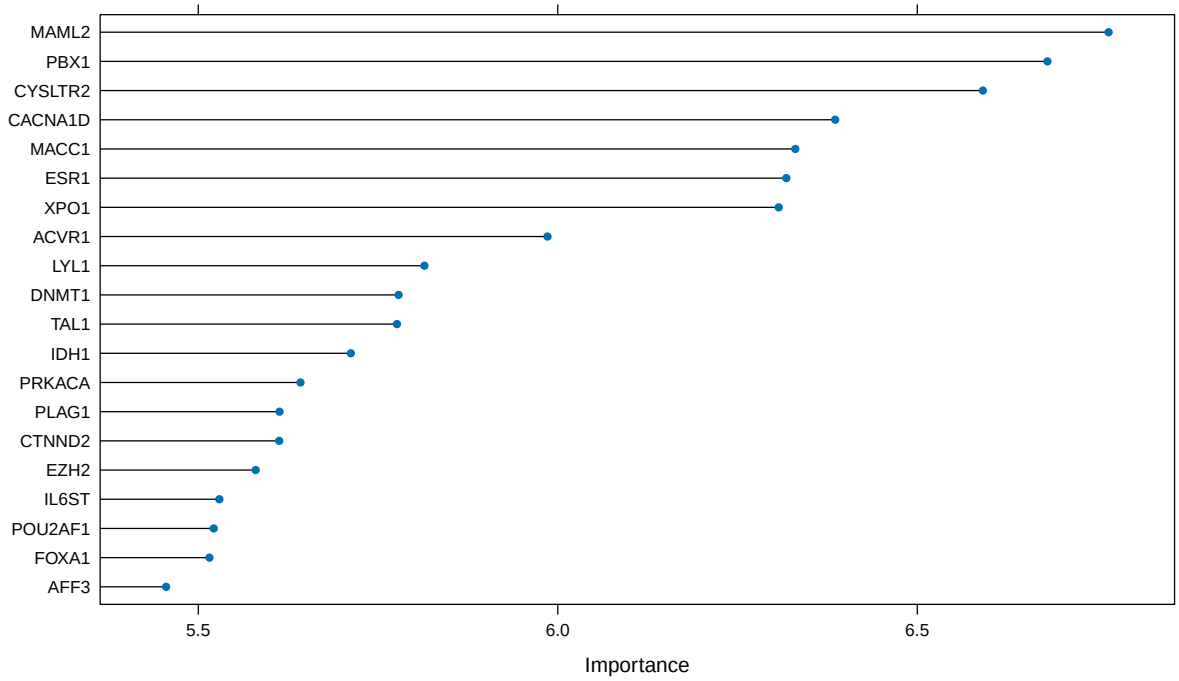
varImp Fase 2

```r
roc_imp_rf <- varImp(rf2_onco, scale = FALSE)
rownames(roc_imp_rf$importance) = sapply(strsplit(rownames(roc_imp_rf$importance), '_'), `[`, 2)
rownames(roc_imp_rf$importance) = oncogenes_only$symbol[match(rownames(roc_imp_rf$importance), oncogenes

top_idx = order(roc_imp_rf$importance$Normal, decreasing = TRUE)[1:20]
top_genes_onco = data.frame(
  Gen = rownames(roc_imp_rf$importance)[top_idx],
  Importancia_Normal = round(roc_imp_rf$importance$Normal[top_idx], 4),
  Importancia_Tumor = round(roc_imp_rf$importance$Tumor[top_idx], 4),
  Direccion = ifelse(roc_imp_rf$importance$Normal[top_idx] > roc_imp_rf$importance$Tumor[top_idx],
                     "↑ Normal", "↑ Tumor")
)
knitr::kable(top_genes_onco, caption = "RF Solo Oncogenes: Top 20 con dirección")
```

Table 5: RF Solo Oncogenes: Top 20 con dirección

Gen	Importancia_Normal	Importancia_Tumor	Direccion
MAML2	6.7661	6.7661	↑ Tumor
PBX1	6.6811	6.6811	↑ Tumor
CYSLTR2	6.5913	6.5913	↑ Tumor
CACNA1D	6.3858	6.3858	↑ Tumor
MACC1	6.3304	6.3304	↑ Tumor
ESR1	6.3179	6.3179	↑ Tumor
XPO1	6.3073	6.3073	↑ Tumor
ACVR1	5.9857	5.9857	↑ Tumor
LYL1	5.8145	5.8145	↑ Tumor
DNMT1	5.7786	5.7786	↑ Tumor
TAL1	5.7764	5.7764	↑ Tumor
IDH1	5.7122	5.7122	↑ Tumor
PRKACA	5.6421	5.6421	↑ Tumor
PLAG1	5.6130	5.6130	↑ Tumor
CTNND2	5.6125	5.6125	↑ Tumor
EZH2	5.5798	5.5798	↑ Tumor
IL6ST	5.5293	5.5293	↑ Tumor
POU2AF1	5.5215	5.5215	↑ Tumor
FOXA1	5.5155	5.5155	↑ Tumor
AFF3	5.4552	5.4552	↑ Tumor

```r
plot(roc_imp_rf, top = 20)
```



Comparación con modelos más complejos

Para comprobar si la elección de Random Forest y SVM está justificada, los comparamos sobre el mismo conjunto de oncogenes con dos modelos más complejos: Gradient Boosting (`gbm`) y una red neuronal (`nnet`), esta última como representante del aprendizaje profundo. Todos se entrenan con la misma validación cruzada de 10 particiones.

```
library(gbm)
library(nnet)

# Gradient Boosting
set.seed(900808)
gbm_onco <- train(formula, data = train_set_f2, method = "gbm",
                  tuneLength = 5, trControl = train_control, metric = "ROC",
                  verbose = FALSE)

# Red neuronal
set.seed(900808)
nnet_onco <- train(formula, data = train_set_f2, method = "nnet",
                   tuneLength = 5, trControl = train_control, metric = "ROC",
                   trace = FALSE, MaxNWts = 10000, maxit = 300)
```

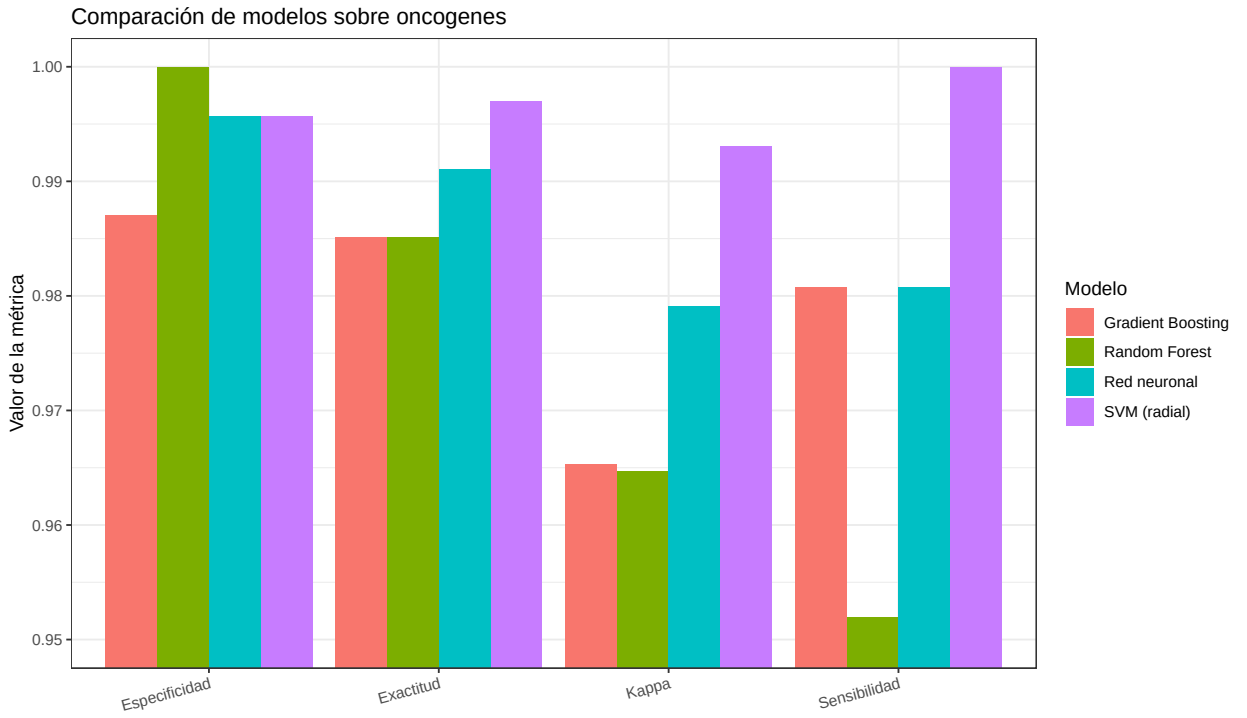
```
# Evaluar los 4 modelos sobre el test set de la fase 2
modelos <- list("Random Forest" = rf2_onco, "SVM (radial)" = svm_onco,
                "Gradient Boosting" = gbm_onco, "Red neuronal" = nnet_onco)

resultados <- lapply(modelos, function(m) {
  pred <- predict(m, newdata = test_set_f2)
```

```r
  cm <- confusionMatrix(pred, test_set_f2$type, positive = "Normal")
  c(Exactitud = cm$overall["Accuracy"],
    Kappa = cm$overall["Kappa"],
    Sensibilidad = cm$byClass["Sensitivity"],
    Especificidad = cm$byClass["Specificity"])
})

tabla_modelos <- as.data.frame(do.call(rbind, resultados))
colnames(tabla_modelos) <- c("Exactitud", "Kappa", "Sensibilidad", "Especificidad")
knitr::kable(tabla_modelos, digits = 4,
             caption = "Comparación de los cuatro modelos sobre el conjunto de oncogenes")
```

Table 6: Comparación de los cuatro modelos sobre el conjunto de oncogenes

	Exactitud	Kappa	Sensibilidad	Especificidad
Random Forest	0.9851	0.9647	0.9519	1.0000
SVM (radial)	0.9970	0.9931	1.0000	0.9957
Gradient Boosting	0.9851	0.9653	0.9808	0.9871
Red neuronal	0.9911	0.9791	0.9808	0.9957

```r
# Pasar la tabla a formato largo (sin dependencias externas) para graficar
tabla_larga <- data.frame(
  Modelo  = rep(rownames(tabla_modelos), times = ncol(tabla_modelos)),
  Metrica = rep(colnames(tabla_modelos), each = nrow(tabla_modelos)),
  Valor   = as.vector(as.matrix(tabla_modelos))
)

p_comp_modelos <- ggplot(tabla_larga, aes(x = Metrica, y = Valor, fill = Modelo)) +
  geom_col(position = "dodge") +
  coord_cartesian(ylim = c(0.95, 1)) +
  labs(title = "Comparación de modelos sobre oncogenes",
       y = "Valor de la métrica", x = "") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 15, hjust = 1))
p_comp_modelos
```




```
ggsave("figuras/comparacion_modelos.png", p_comp_modelos, width = 10, height = 6, dpi = 300)
```

Conclusión: la SVM es el mejor modelo en exactitud, kappa y sensibilidad, clasificando correctamente todas las muestras normales (la clase minoritaria); solo el Random Forest la supera ligeramente en especificidad, al no clasificar ningún tumor como normal. Ni el Gradient Boosting ni la red neuronal superan a la SVM, lo que respalda centrar el análisis en modelos simples e interpretables: en estos datos no hay que renunciar a rendimiento para ganar interpretabilidad.

Fase 3: Explicabilidad (XAI)

En esta fase aplicamos tres técnicas de IA explicable sobre los modelos de la Fase 2 (solo oncogenes): LIME, SHAP y PDP.

LIME: Explicaciones locales

LIME (Local Interpretable Model-agnostic Explanations) genera explicaciones para predicciones individuales. Para cada paciente, identifica los genes que más influyeron en la decisión del modelo. Nos centramos en los casos más informativos: las cinco muestras sanas que el Random Forest clasifica incorrectamente como tumorales, que son sus únicos errores en el conjunto de prueba.

```
# Usar los datos de la Fase 2 (oncogenes) para la explicabilidad
train_set <- train_set_f2
test_set <- test_set_f2
```

```
library(lime)
library(iml)
```

```r
library(pdp)

# Función para traducir los códigos g_<entrez> a símbolos de gen en las
# explicaciones de LIME (equivale a relabel_genes). Procesa los códigos más
# largos primero para evitar coincidencias parciales.
relabel_genes <- function(expl) {
  codes <- unique(expl$feature)
  codes <- codes[order(nchar(codes), decreasing = TRUE)]
  for (code in codes) {
    entrez <- sub("^g_", "", code)
    sym <- oncogenes_only$symbol[match(entrez, oncogenes_only$entrez)]
    if (!is.na(sym)) {
      expl$feature      <- gsub(code, sym, expl$feature, fixed = TRUE)
      expl$feature_desc <- gsub(code, sym, expl$feature_desc, fixed = TRUE)
    }
  }
  expl
}

# Crear explainers
explainer_rf <- lime(
  x = train_set[, -ncol(train_set)],
  model = rf2_onco
)

explainer_svm <- lime(
  x = train_set[, -ncol(train_set)],
  model = svm_onco
)
```

Selección de los casos difíciles

En lugar de elegir casos al azar, seleccionamos las muestras sanas que el modelo clasifica mal: son los casos fronterizos donde mejor se ve cómo decide.

```r
predictions_rf <- predict(rf2_onco, newdata = test_set)

correctos   <- which(predictions_rf == test_set$type)
incorrectos <- which(predictions_rf != test_set$type)
cat("Correctos:", length(correctos), " | Incorrectos:", length(incorrectos), "\n")
```

```
## Correctos: 331  | Incorrectos: 5
```

```r
# Las muestras NORMALES mal clasificadas como tumorales (los unicos errores del RF)
indices_a_explicar <- which(predictions_rf != test_set$type & test_set$type == "Normal")

cat("Casos a explicar (normales mal clasificadas):", length(indices_a_explicar), "\n")
```

```
## Casos a explicar (normales mal clasificadas): 5
```

```r
cat("Muestras:", rownames(test_set)[indices_a_explicar], "\n")
```

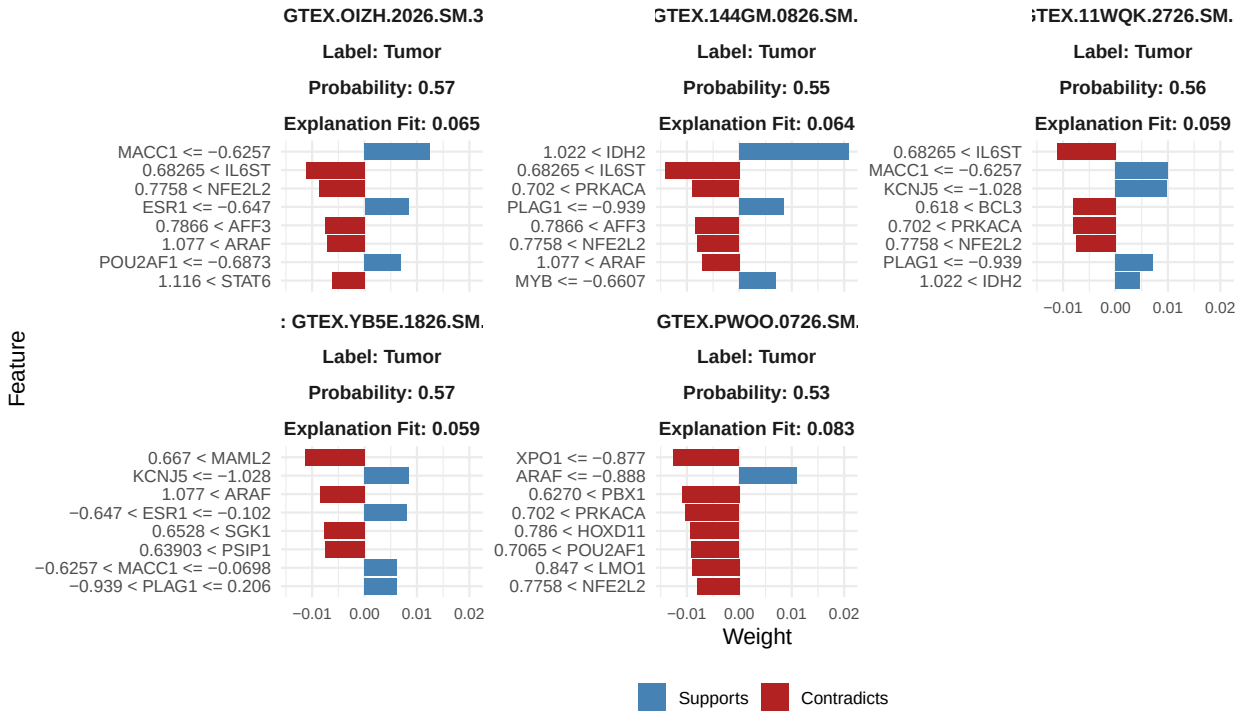
```
## Muestras: GTEX.OIZH.2026.SM.3NB1M GTEX.144GM.0826.SM.5O98R GTEX.11WQK.2726.SM.5EQMU GTEX.YB5E.1826.SI
```

```r
cat("Predicciones:", as.character(predictions_rf[indices_a_explicar]), "\n")
```

```
## Predicciones: Tumor Tumor Tumor Tumor Tumor
```

LIME: Random Forest

```r
set.seed(42)  # Semilla fija para que las figuras no cambien al re-knitear
explanation_rf <- lime::explain(
  x = test_set[indices_a_explicar, -ncol(test_set)],
  explainer = explainer_rf,
  n_labels = 1,
  n_features = 8
)

# Tema legible comun a todas las figuras LIME
tema_lime <- ggplot2::theme_minimal(base_size = 13) +
  ggplot2::theme(axis.text.y = ggplot2::element_text(size = 10),
                 axis.text.x = ggplot2::element_text(size = 9),
                 strip.text  = ggplot2::element_text(size = 11, face = "bold"),
                 legend.position = "bottom")

# Etiquetas de cada panel con el id de muestra (para identificarlas)
p_lime_rf <- plot_features(relabel_genes(explanation_rf), ncol = 3) + tema_lime
p_lime_rf
```

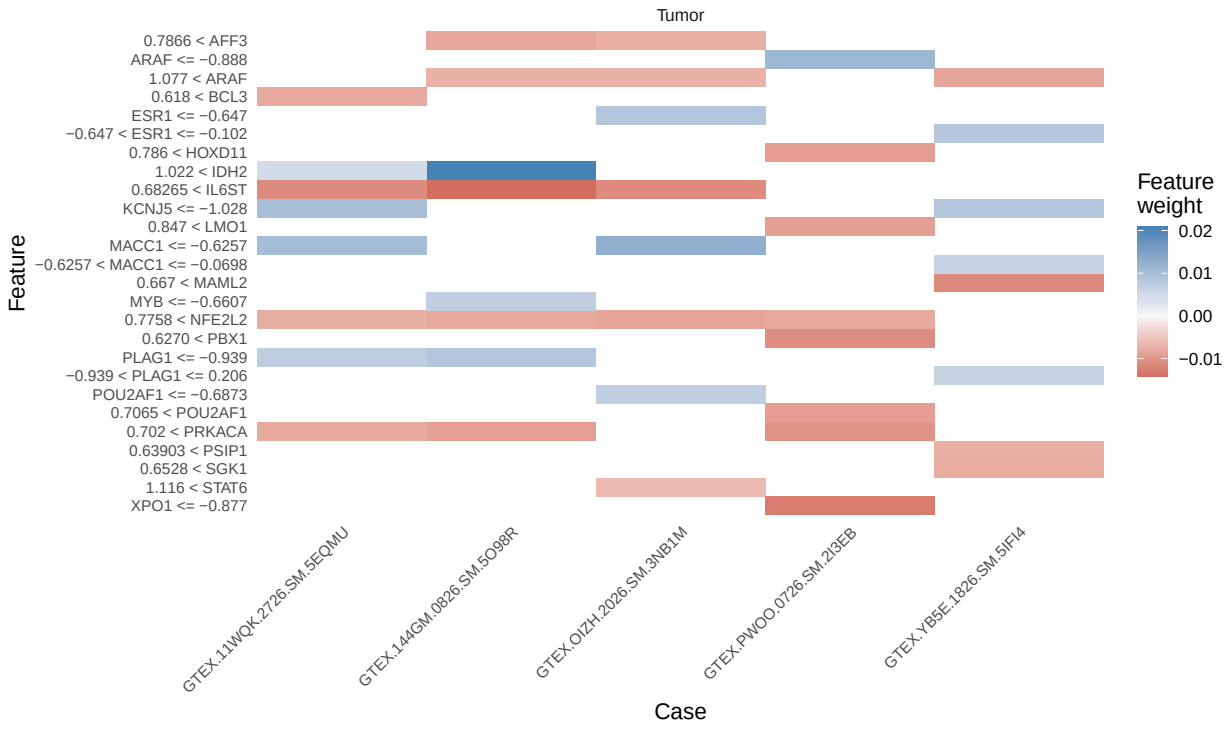



```
ggsave("figuras/lime_rf.png", p_lime_rf, width = 12, height = 7.5, dpi = 300, bg = "white")
```

Las barras azules (Supports) indican genes que apoyan la predicción del modelo. Las rojas (Contradicts) van en contra. La longitud indica cuánto influye cada gen.

LIME: Heatmap

```
p_heat <- plot_explanations(relabel_genes(explanation_rf)) +
  ggplot2::theme_minimal(base_size = 13) +
  ggplot2::theme(axis.text.y = ggplot2::element_text(size = 9),
                 axis.text.x = ggplot2::element_text(size = 9, angle = 45, hjust = 1),
                 panel.grid = ggplot2::element_blank()) +
  ggplot2::labs(fill = "Peso")
p_heat
```

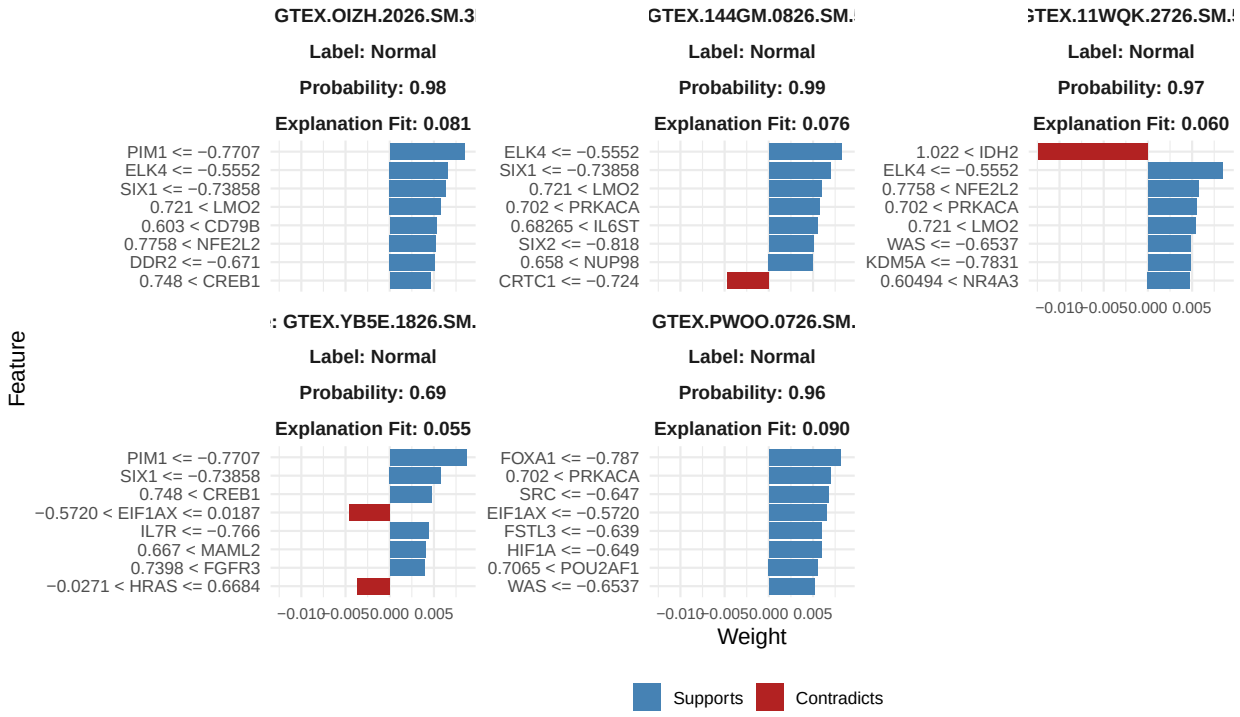



```
ggsave("figuras/lime_heatmap.png", p_heat, width = 9, height = 7, dpi = 300, bg = "white")
```

El heatmap muestra todos los pacientes a la vez. Azul = apoya la predicción, rojo = contradice. Permite identificar patrones comunes.

LIME: SVM (comparación con RF)

```
set.seed(42)
explanation_svm <- lime::explain(
  x = test_set[indices_a_explicar, -ncol(test_set)],
  explainer = explainer_svm,
  n_labels = 1,
  n_features = 8
)

p_lime_svm <- plot_features(relabel_genes(explanation_svm), ncol = 3) + tema_lime
p_lime_svm
```




```
ggsave("figuras/lime_svm.png", p_lime_svm, width = 12, height = 7.5, dpi = 300, bg = "white")
```

Explicaciones individuales (Anexo D)

Guardamos cada uno de los cinco casos por separado, en Random Forest y en SVM, para incluirlos en el anexo.

```
dir.create("figuras/anexo", showWarnings = FALSE, recursive = TRUE)
exp_rf_t  <- relabel_genes(explanation_rf)
exp_svm_t <- relabel_genes(explanation_svm)
muestras  <- unique(exp_rf_t$case)

for (i in seq_along(muestras)) {
  ggsave(sprintf("figuras/anexo/fallo_rf_%d.png", i),
         plot_features(exp_rf_t[exp_rf_t$case == muestras[i], ]) + tema_lime,
         width = 7, height = 5, dpi = 300, bg = "white")
  ggsave(sprintf("figuras/anexo/fallo_svm_%d.png", i),
         plot_features(exp_svm_t[exp_svm_t$case == muestras[i], ]) + tema_lime,
         width = 7, height = 5, dpi = 300, bg = "white")
}
```

Traducción de nombres de genes

```
genes_lime <- unique(explanation_rf$feature)
entrez_ids <- sapply(strsplit(genes_lime, '_'), `[`, 2)
gene_names <- oncogenes_only$symbol[match(entrez_ids, oncogenes_only$entrez)]
```

```r
traduccion <- data.frame(codigo = genes_lime, gen = gene_names)
traduccion <- traduccion[!is.na(traduccion$gen),]
knitr::kable(traduccion, caption = "Traducción de códigos a nombres de oncogenes (LIME)")
```

Table 7: Traducción de códigos a nombres de oncogenes (LIME)

codigo	gen
g_346389	MACC1
g_3572	IL6ST
g_4780	NFE2L2
g_2099	ESR1
g_3899	AFF3
g_5450	POU2AF1
g_369	ARAF
g_6778	STAT6
g_3418	IDH2
g_5324	PLAG1
g_5566	PRKACA
g_4602	MYB
g_3762	KCNJ5
g_602	BCL3
g_84441	MAML2
g_11168	PSIP1
g_6446	SGK1
g_7514	XPO1
g_5087	PBX1
g_3237	HOXD11
g_4004	LMO1

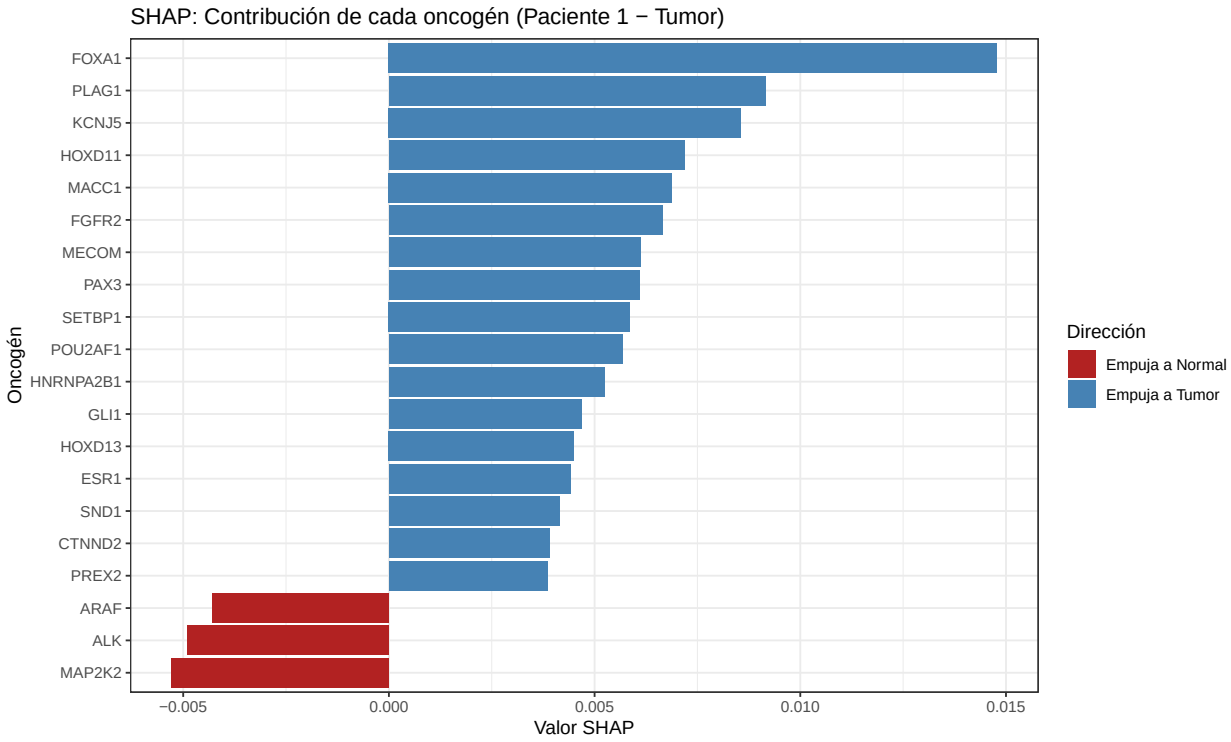
SHAP: Valores de Shapley

SHAP (SHapley Additive exPlanations) calcula la contribución exacta de cada gen a cada predicción individual, basándose en los valores de Shapley de teoría de juegos.

SHAP: Paciente tumoral

```r
predictor_rf <- Predictor$new(
  model = rf2_onco,
  data = train_set,
  y = "type",
  type = "prob"
)

shapley_1 <- Shapley$new(
  predictor = predictor_rf,
  x.interest = test_set[1, ]
)

shap_results <- shapley_1$results
```

```r
shap_results$entrez <- sapply(strsplit(as.character(shap_results$feature), '_'), `[`, 2)
shap_results$gen <- oncogenes_only$symbol[match(shap_results$entrez, oncogenes_only$entrez)]
shap_results$gen[is.na(shap_results$gen)] <- shap_results$feature[is.na(shap_results$gen)]

shap_tumor <- subset(shap_results, class == "Tumor")
shap_tumor <- shap_tumor[order(abs(shap_tumor$phi), decreasing = TRUE),]
top20_shap <- head(shap_tumor, 20)

p_shap_tumor <- ggplot(top20_shap, aes(x = reorder(gen, phi), y = phi)) +
  geom_bar(stat = "identity", aes(fill = phi > 0)) +
  scale_fill_manual(values = c("TRUE" = "steelblue", "FALSE" = "firebrick"),
                    labels = c("TRUE" = "Empuja a Tumor", "FALSE" = "Empuja a Normal"),
                    name = "Dirección") +
  coord_flip() +
  labs(x = "Oncogén", y = "Valor SHAP",
       title = "SHAP: Contribución de cada oncogén (Paciente 1 - Tumor)") +
  theme_bw()
p_shap_tumor
```




```r
ggsave("figuras/shap_paciente_tumor.png", p_shap_tumor, width = 9, height = 7, dpi = 300)

cat("Paciente 1 - Tipo real:", as.character(test_set$type[1]), "\n")
```

```
## Paciente 1 - Tipo real: Tumor
```

```r
cat("Top 5 genes que empujan a Tumor:\n")
```

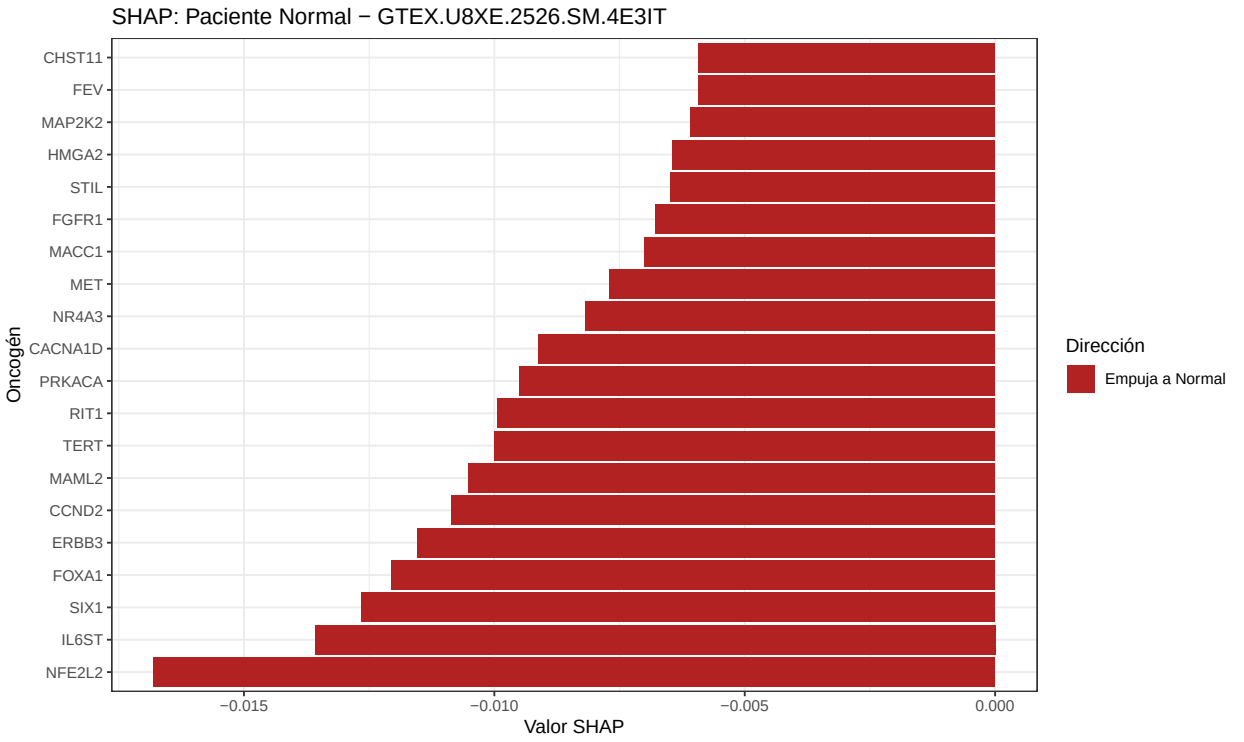
```
## Top 5 genes que empujan a Tumor:
```

```r
print(head(subset(top20_shap, phi > 0, select = c("gen", "phi")), 5))
```

```
##        gen     phi
## 419  FOXA1 0.01478
## 297  PLAG1 0.00916
## 317  KCNJ5 0.00856
## 330 HOXD11 0.00718
## 367  MACC1 0.00688
```

SHAP: Paciente normal

```r
idx_normal <- which(test_set$type == "Normal")[1]

shapley_normal <- Shapley$new(
  predictor = predictor_rf,
  x.interest = test_set[idx_normal, ]
)

shap_res_n <- shapley_normal$results
shap_res_n$entrez <- sapply(strsplit(as.character(shap_res_n$feature), '_'), `[`, 2)
shap_res_n$gen <- oncogenes_only$symbol[match(shap_res_n$entrez, oncogenes_only$entrez)]
shap_res_n$gen[is.na(shap_res_n$gen)] <- shap_res_n$feature[is.na(shap_res_n$gen)]

shap_tumor_n <- subset(shap_res_n, class == "Tumor")
shap_tumor_n <- shap_tumor_n[order(abs(shap_tumor_n$phi), decreasing = TRUE),]
top20_normal <- head(shap_tumor_n, 20)

p_shap_normal <- ggplot(top20_normal, aes(x = reorder(gen, phi), y = phi)) +
  geom_bar(stat = "identity", aes(fill = phi > 0)) +
  scale_fill_manual(values = c("TRUE" = "steelblue", "FALSE" = "firebrick"),
                    labels = c("TRUE" = "Empuja a Tumor", "FALSE" = "Empuja a Normal"),
                    name = "Dirección") +
  coord_flip() +
  labs(x = "Oncogén", y = "Valor SHAP",
       title = paste("SHAP: Paciente Normal -", rownames(test_set)[idx_normal])) +
  theme_bw()
p_shap_normal
```




```
ggsave("figuras/shap_paciente_normal.png", p_shap_normal, width = 9, height = 7, dpi = 300)

cat("Top 5 genes que empujan a Normal:\n")
```

```
## Top 5 genes que empujan a Normal:
```

```
print(head(subset(top20_normal, phi < 0, select = c("gen", "phi")), 5))
```

```
##        gen      phi
## 467 NFE2L2 -0.01680
## 370  IL6ST -0.01358
## 433   SIX1 -0.01266
## 419  FOXA1 -0.01206
## 257  ERBB3 -0.01154
```

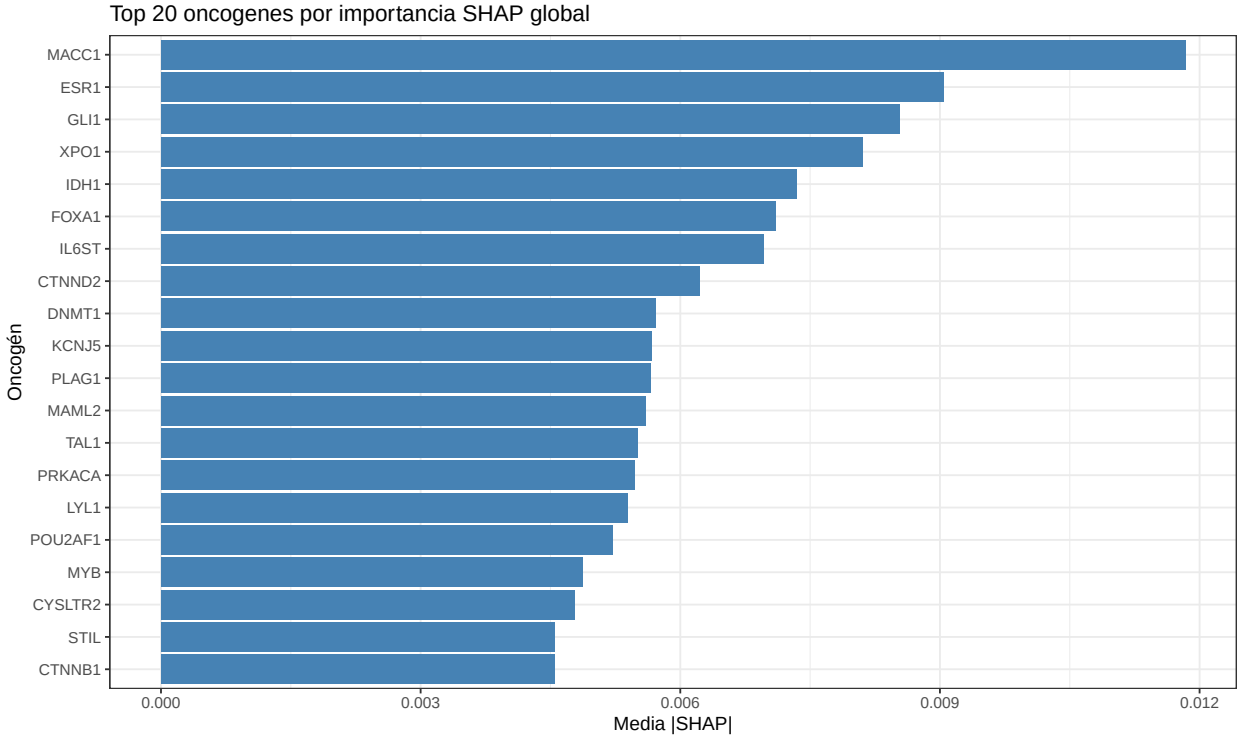
Comparando ambos gráficos se observa que FOXA1 aparece como gen importante en ambos pacientes pero en direcciones opuestas: empuja hacia Tumor en el paciente tumoral y hacia Normal en el paciente sano.

SHAP global: 20 pacientes

Calculamos SHAP para 20 pacientes (10 Tumor + 10 Normal) para obtener un ranking global de importancia.

```
set.seed(42)
idx_tumor <- sample(which(test_set$type == "Tumor"), 10)
idx_normal_g <- sample(which(test_set$type == "Normal"), 10)
idx_shap <- c(idx_tumor, idx_normal_g)
```

```r
shap_results_all <- list()
for(i in seq_along(idx_shap)) {
  shap_results_all[[i]] <- Shapley$new(
    predictor = predictor_rf,
    x.interest = test_set[idx_shap[i], ]
  )$results
  shap_results_all[[i]]$paciente <- i
  shap_results_all[[i]]$tipo_real <- as.character(test_set$type[idx_shap[i]])
}

shap_all <- do.call(rbind, shap_results_all)

shap_all$entrez <- sapply(strsplit(as.character(shap_all$feature), '_'), `[`, 2)
shap_all$gen <- oncogenes_only$symbol[match(shap_all$entrez, oncogenes_only$entrez)]

shap_tumor_all <- subset(shap_all, class == "Tumor")
shap_importancia <- aggregate(abs(phi) ~ gen, data = shap_tumor_all, FUN = mean)
colnames(shap_importancia) <- c("gen", "mean_abs_shap")
shap_importancia <- shap_importancia[order(shap_importancia$mean_abs_shap, decreasing = TRUE),]
shap_importancia <- shap_importancia[!is.na(shap_importancia$gen),]

top20_shap_global <- head(shap_importancia, 20)

p_shap_global <- ggplot(top20_shap_global, aes(x = reorder(gen, mean_abs_shap), y = mean_abs_shap)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  coord_flip() +
  labs(x = "Oncogén", y = "Media |SHAP|", title = "Top 20 oncogenes por importancia SHAP global") +
  theme_bw()
p_shap_global
```



```
ggsave("figuras/shap_global.png", p_shap_global, width = 9, height = 7, dpi = 300)

knitr::kable(top20_shap_global, digits = 4, caption = "Top 20 oncogenes por importancia SHAP global")
```

Table 8: Top 20 oncogenes por importancia SHAP global

	gen	mean_abs_shap
135	MACC1	0.0118
71	ESR1	0.0090
93	GLI1	0.0085
251	XPO1	0.0081
111	IDH1	0.0073
87	FOXA1	0.0071
114	IL6ST	0.0070
54	CTNND2	0.0062
63	DNMT1	0.0057
122	KCNJ5	0.0057
184	PLAG1	0.0057
139	MAML2	0.0056
230	TAL1	0.0055
192	PRKACA	0.0055
134	LYL1	0.0054
186	POU2AF1	0.0052
154	MYB	0.0049
56	CYSLTR2	0.0048
227	STIL	0.0045
53	CTNNB1	0.0045

Comparación varImp vs SHAP

```
roc_imp <- varImp(rf2_onco, scale = FALSE)
rownames(roc_imp$importance) <- sapply(strsplit(rownames(roc_imp$importance), '_'), `[`, 2)
rownames(roc_imp$importance) <- oncogenes_only$symbol[match(rownames(roc_imp$importance), oncogenes_only
top20_varimp <- rownames(roc_imp$importance[order(roc_imp$importance$Normal, decreasing = T),])[1:20]

comunes <- intersect(top20_varimp, top20_shap_global$gen)
solo_varimp <- setdiff(top20_varimp, top20_shap_global$gen)
solo_shap <- setdiff(top20_shap_global$gen, top20_varimp)

comparacion_tecnicas <- data.frame(
  Metrica = c("Genes en Top 20 varImp", "Genes en Top 20 SHAP", "Genes comunes", "Solo en varImp", "Solo
  Valor = c(20, 20, length(comunes), length(solo_varimp), length(solo_shap)),
  Genes = c(
    paste(top20_varimp, collapse = ", "),
    paste(top20_shap_global$gen, collapse = ", "),
    paste(comunes, collapse = ", "),
    paste(solo_varimp, collapse = ", "),
    paste(solo_shap, collapse = ", ")
  )
)
knitr::kable(comparacion_tecnicas, caption = "Comparación varImp vs SHAP")
```

Table 9: Comparación varImp vs SHAP

Metrica	Valor	Genes
Genes en Top 20 varImp	20	MAML2, PBX1, CYSLTR2, CACNA1D, MACC1, ESR1, XPO1, ACVR1, LYL1, DNMT1, TAL1, IDH1, PRKACA, PLAG1, CTNND2, EZH2, IL6ST, POU2AF1, FOXA1, AFF3
Genes en Top 20 SHAP	20	MACC1, ESR1, GLI1, XPO1, IDH1, FOXA1, IL6ST, CTNND2, DNMT1, KCNJ5, PLAG1, MAML2, TAL1, PRKACA, LYL1, POU2AF1, MYB, CYSLTR2, STIL, CTNNB1
Genes comunes	15	MAML2, CYSLTR2, MACC1, ESR1, XPO1, LYL1, DNMT1, TAL1, IDH1, PRKACA, PLAG1, CTNND2, IL6ST, POU2AF1, FOXA1
Solo en varImp	5	PBX1, CACNA1D, ACVR1, EZH2, AFF3
Solo en SHAP	5	GLI1, KCNJ5, MYB, STIL, CTNNB1

Se observa un **75% de coincidencia** (15 de 20 genes). Esto indica que varImp identifica correctamente la mayoría de genes importantes, pero SHAP revela genes adicionales y además proporciona información que varImp no puede: la dirección y magnitud de la contribución por paciente individual.

PDP: Partial Dependence Plots

Los PDP muestran cómo cambia la predicción cuando varía la expresión de un gen concreto, manteniendo el resto constantes.

PDP: Top 5 oncogenes

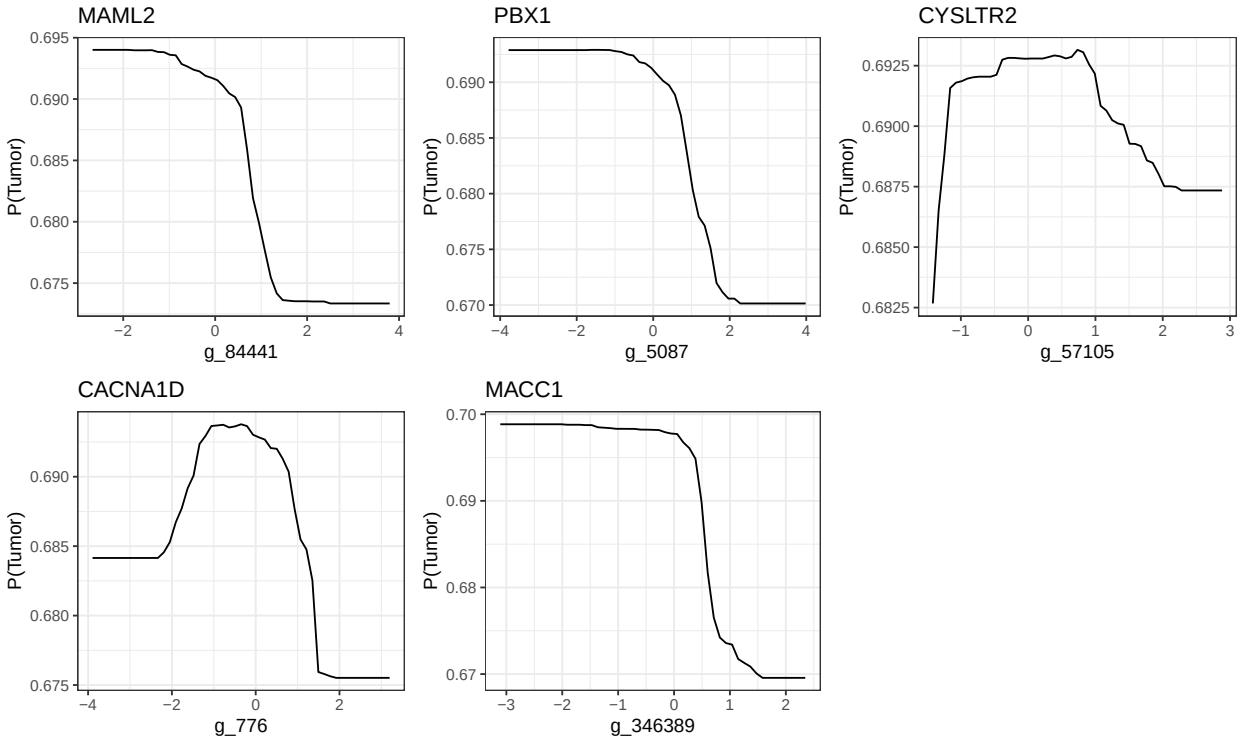
```
roc_imp_rf <- varImp(rf2_onco, scale = FALSE)
top5_idx <- order(roc_imp_rf$importance$Normal, decreasing = TRUE)[1:5]
top5_genes <- rownames(roc_imp_rf$importance)[top5_idx]

entrez_ids <- sapply(strsplit(top5_genes, '_'), `[`, 2)
nombres_top5 <- oncogenes_only$symbol[match(entrez_ids, oncogenes_only$entrez)]

knitr::kable(data.frame(Codigo = top5_genes, Oncogen = nombres_top5), caption = "Top 5 genes para PDP")
```

Table 10: Top 5 genes para PDP

Codigo	Oncogen
g__84441	MAML2
g__5087	PBX1
g__57105	CYSLTR2
g__776	CACNA1D
g__346389	MACC1

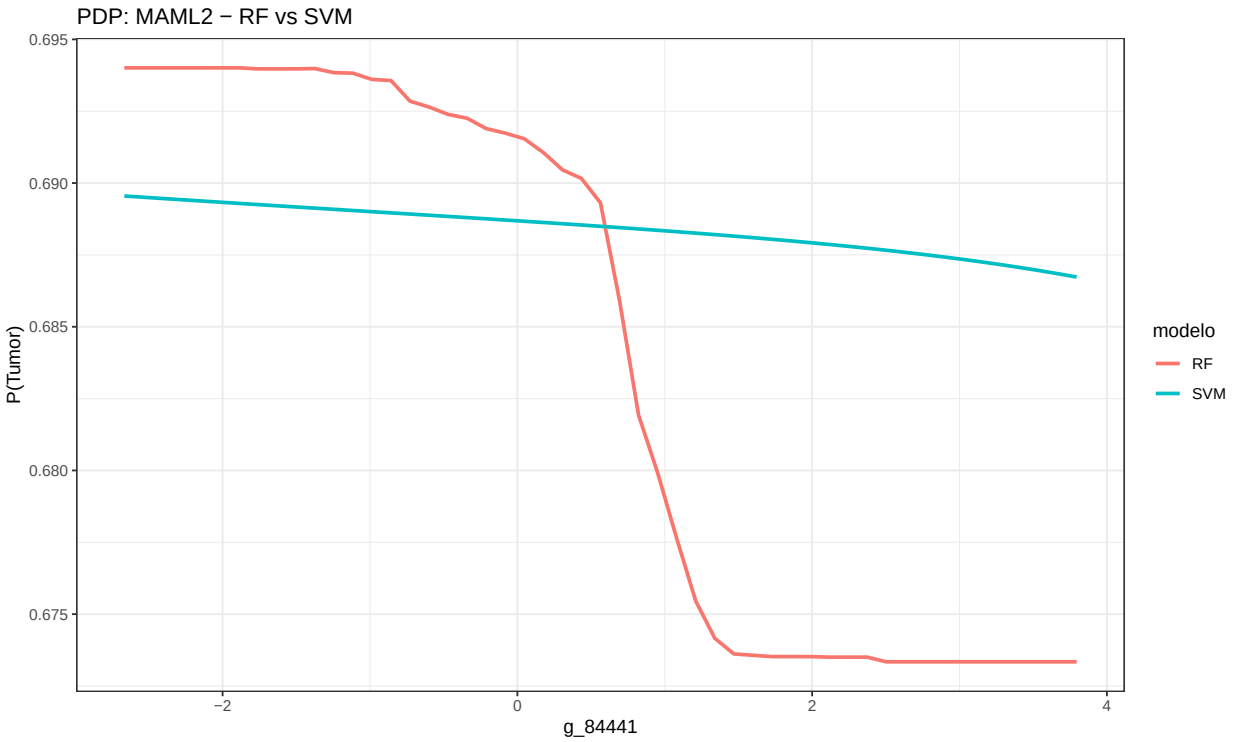
```
# Generar un PDP por gen con autoplot (ggplot) y combinarlos con gridExtra.
# (Esto sustituye al antiguo par(mfrow), que lattice/plotPartial no respeta.)
pdp_plots <- lapply(1:5, function(i) {
```

```
  pd <- partial(rf2_onco, pred.var = top5_genes[i], prob = TRUE, which.class = "Tumor")
  autoplot(pd) + ggtitle(nombres_top5[i]) + ylab("P(Tumor)") + theme_bw()
})

pdp_arr <- arrangeGrob(grobs = pdp_plots, ncol = 3)
ggsave("figuras/pdp_top5.png", pdp_arr, width = 12, height = 7, dpi = 300)
grid.arrange(grobs = pdp_plots, ncol = 3)
```



PDP: Comparación RF vs SVM

```
pd_rf <- partial(rf2_onco, pred.var = top5_genes[1], prob = TRUE, which.class = "Tumor")
pd_svm <- partial(svm_onco, pred.var = top5_genes[1], prob = TRUE, which.class = "Tumor")

pd_rf$modelo <- "RF"
pd_svm$modelo <- "SVM"
pd_both <- rbind(pd_rf, pd_svm)

p_pdp_comp <- ggplot(pd_both, aes(x = .data[[top5_genes[1]]], y = yhat, color = modelo)) +
  geom_line(linewidth = 1) +
  labs(y = "P(Tumor)", title = paste("PDP:", nombres_top5[1], "- RF vs SVM")) +
  theme_bw()
p_pdp_comp
```




```
ggsave("figuras/pdp_comparacion.png", p_pdp_comp, width = 9, height = 6, dpi = 300)
```

Ambos modelos muestran la misma tendencia: a mayor expresión de MAML2, menor probabilidad de tumor. Sin embargo, RF presenta una caída más abrupta (un umbral) mientras que SVM muestra una transición más gradual.

Aplicación interactiva (Shiny)

Como extensión del trabajo, se ha desarrollado una aplicación web con Shiny que pone la clasificación y la explicabilidad al alcance de un usuario sin conocimientos de R. La aplicación permite elegir un modelo (Random Forest o SVM) y una muestra del conjunto de prueba, y devuelve la predicción (tumoral o sana), si coincide con la etiqueta real, y una explicación LIME con los oncogenes que más han influido en esa decisión, mostrados con su nombre real.

La app no forma parte de este notebook: vive en el archivo `app.R` del repositorio y se ejecuta con el botón "Run App" de RStudio. Al arrancar, carga los modelos y los datos necesarios (los dos modelos de la Fase 2, los conjuntos de entrenamiento y prueba y la tabla de oncogenes) desde el archivo `datos_app.RData`. Es la forma de convertir el análisis en una herramienta usable, alineada con el objetivo de interpretabilidad de este trabajo.

Conclusiones

1. **Fase 1:** Los modelos RF y SVM clasifican muestras tumorales vs normales con accuracy >97%, y PRAD vs LUAD con accuracy del 100%.
2. **Fase 2:** Usando solo oncogenes (253 genes en vez de 506), el rendimiento se mantiene o mejora ligeramente, justificando centrar el análisis en oncogenes.

3. **Comparación de modelos:** frente a Gradient Boosting y una red neuronal, la SVM resulta el mejor modelo en exactitud, kappa y sensibilidad, clasificando correctamente todas las muestras normales; solo el Random Forest la supera ligeramente en especificidad, al clasificar todos los tumores sin error. Los modelos más complejos no mejoran su rendimiento, lo que respalda el uso de modelos simples e interpretables.
4. **LIME:** Permite explicar predicciones individuales. Se identificó que el RF mal clasifica 5 muestras normales como tumorales con probabilidad baja (~55%), mientras que el SVM las clasifica correctamente con alta confianza (~98%).
5. **SHAP:** FOXA1, KCNJ5 y PLAG1 son los oncogenes que más empujan las predicciones tumorales. NFE2L2, FOXA1 e IL6ST son los que más empujan hacia normal. El 75% de los genes del ranking SHAP coincide con varImp.
6. **PDP:** Los oncogenes top muestran relaciones no lineales con umbrales. RF y SVM coinciden en la dirección pero difieren en la forma de la transición.

```
sessionInfo()
```

```
## R version 4.5.3 (2026-03-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Spanish_Spain.utf8  LC_CTYPE=Spanish_Spain.utf8
## [3] LC_MONETARY=Spanish_Spain.utf8 LC_NUMERIC=C
## [5] LC_TIME=Spanish_Spain.utf8
##
## time zone: Europe/Madrid
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] nnet_7.3-20          gbm_2.2.3            pdp_0.8.3
##  [4] iml_0.11.4           lime_0.5.4           Rtsne_0.17
##  [7] kernlab_0.9-33       e1071_1.7-17         gridExtra_2.3
## [10] randomForest_4.7-1.2 caret_7.0-1          lattice_0.22-9
## [13] ggplot2_4.0.2
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1     timeDate_4052.112    dplyr_1.2.1
##  [4] farver_2.1.2         S7_0.2.1             fastmap_1.2.0
##  [7] pROC_1.19.0.1        digest_0.6.39        rpart_4.1.24
## [10] timechange_0.4.0     lifecycle_1.0.5      survival_3.8-6
## [13] magrittr_2.0.5       compiler_4.5.3       rlang_1.2.0
## [16] tools_4.5.3          yaml_2.3.12          data.table_1.18.2.1
## [19] knitr_1.51           labeling_0.4.3       plyr_1.8.9
## [22] RColorBrewer_1.1-3   withr_3.0.2          purrr_1.2.2
## [25] Metrics_0.1.4        grid_4.5.3           stats4_4.5.3
## [28] future_1.70.0        globals_0.19.1       scales_1.4.0
```

```
## [31] iterators_1.0.14     MASS_7.3-65          cli_3.6.6
## [34] rmarkdown_2.31       generics_0.1.4       otel_0.2.0
## [37] future.apply_1.20.2  reshape2_1.4.5       proxy_0.4-29
## [40] stringr_1.6.0        splines_4.5.3        assertthat_0.2.1
## [43] parallel_4.5.3       vctrs_0.7.3          hardhat_1.4.3
## [46] glmnet_5.0           Matrix_1.7-4         listenv_0.10.1
## [49] foreach_1.5.2        gower_1.0.2          recipes_1.3.2
## [52] glue_1.8.0           parallelly_1.46.1    codetools_0.2-20
## [55] lubridate_1.9.5      stringi_1.8.7        gtable_0.3.6
## [58] shape_1.4.6.1        tibble_3.3.1         pillar_1.11.1
## [61] htmltools_0.5.9      ipred_0.9-15         lava_1.9.0
## [64] R6_2.6.1             evaluate_1.0.5       backports_1.5.1
## [67] class_7.3-23         Rcpp_1.1.1           nlme_3.1-168
## [70] prodlim_2026.03.11   checkmate_2.3.4      xfun_0.57
## [73] ModelMetrics_1.2.2.2 pkgconfig_2.0.3
```